SMASH: Scalable Maliciously Secure Hybrid Multi-party Computation Framework for Privacy-Preserving Large Language Models

Yunlv Lv[1,2,3], *Rui Zhang*[*1,2,3], *Zhiyuan Zhang*[4], *Ziyi Wan*[1,2,3], *Lanxue Zhang*[1,3], *Minhui Xue*[5], *Jiangtao Li*[6], *and Yanan Cao*[1,3]

[1]*Institute of Information Engineering, Chinese Academy of Sciences, China*
[2]*State Key Laboratory of Cyberspace Security Defense, China*
[3]*School of Cyber Security, University of Chinese Academy of Sciences, China*
[4]*Max Planck Institute for Security and Privacy, Germany*
[5]*CSIRO's Data61 and Responsible AI Research (RAIR) Centre, Adelaide University, Australia*
[6]*East China Normal University, China*

Abstract

The meteoric rise of Large Language Models (LLMs) has sparked an urgent need for privacy-preserving inference. However, existing maliciously secure multi-party computation (MPC) frameworks face a "performance collapse" when scaling to large models, primarily due to the quadratic ($O(n^2)$) communication overhead of nonlinear operators and expensive share conversions. This paper presents SMASH, a highly scalable, maliciously secure hybrid MPC framework that shatters these bottlenecks. SMASH introduces a novel *DFT-based rotation* technique and a lightweight *zero-knowledge proof of knowledge (ZKPoK)* construction to evaluate nonlinear operations. This approach achieves linear communication complexity ($O(n)$) relative to the party count, independent of function complexity. Furthermore, SMASH provides efficient protocols for maliciously secure A2L/L2A conversions between arithmetic and LUT shares with low overhead, and optimized A2B/B2A for arithmetic-Boolean bridging implemented as an SM-LUT-inspired semi-honest variant. Extensive benchmarks demonstrate that SMASH outperforms state-of-the-art frameworks (e.g., MP-SPDZ, MD-ML) by up to 18.9× in runtime and achieves a communication reduction of up to $10^3\times$. With its constant-round online phase and low WAN sensitivity, SMASH paves the way for secure, geographically distributed LLM deployments, achieving a balance between adversarial robustness and practical efficiency.

1 Introduction

Consider, for example, a large infectious-disease response (e.g., COVID-19 or a future pandemic). Such a lifecycle splits naturally into stages with distinct multi-party needs in a distributed manner: (1) surveillance and early warning, which aggregates signals from hundreds of hospitals, public-health agencies, and labs such as EHR summaries, daily case counts, clinical notes, genomic sequences, and social-media indicators to detect emergent hotspots; (2) diagnostics and model training, where diagnostic models are jointly trained from distributed imaging and lab data held by dozens of hospitals; (3) treatment and prognosis, which combines outcomes, treatment protocols, and genomic markers from multi-center cohorts to refine risk models; and (4) policy and audit, which shares aggregated analytics for cross-jurisdictional decision making while preserving patient privacy. Real examples show these scales are realistic: the 4CE consortium aggregated multi-site EHR analyses across on the order of 100 hospitals during COVID-19 [57], and federated imaging efforts (e.g., UCADI) coordinated 23 hospitals for CT diagnostic models [1]. Similarly, cross-industry pilots (e.g., Swift+Google for federated anti-fraud) and banking pilots illustrate the parallel need in finance for multi-bank collaboration [10,15,28,29]. These scenarios show that dozens to potentially hundreds of distinct organizations may legitimately need to collaborate on ML/LLM tasks in a privacy-sensitive manner.

Hybrid MPC frameworks enable privacy-preserving LLMs (PP-LLMs) by allowing multiple parties to jointly compute functions over their private inputs while preserving confidentiality. These frameworks combine cryptographic techniques, e.g., arithmetic secret sharing, Boolean secret sharing, and homomorphic encryption, tailored to the structure and security requirements of the task. Such architectures align with best practices from agencies like CISA, which identify MPC as a core tool for secure AI deployment [14]. Notably, real-world deployment (i.e., collaborative fraud detection among rival banks, cross-border governance, or healthcare consortia) demands malicious security to guarantee correctness against arbitrary adversarial behavior. While efficient semi-honest systems [35,40] have emerged, extending these to malicious models [16,59,61]) remains a critical challenge.

Hybrid MPC has benefited from the rapid progress of two-party computation protocols, such as ABY [22], Gazelle [34], Chameleon [52], and the constructions of Nikolaenko et al. [46] and Liu et al. [39], which integrated garbled circuits, secret sharing, and homomorphic encryption for mixed Boolean-arithmetic operations, to scalable machine learning frameworks. SecureML [43] enabled two-party training;

*Corresponding author: r-zhang@iie.ac.cn.

Table 1: Comparison of SMASH with Prior MPC Frameworks for PP-LLM

Features / Framework	SMASH (This work)	MD-ML [59] (USENIX'24)	MD-SONIC [61] (TIFS'25)	Garg et al. [27] (SP'24)	SHARK [31] (SP'25)	SHAFT [35] (NDSS'25)	BumbleBee [40] (NDSS'25)	Bolt [48] (SP'24)
Security Model	Malicious	Malicious	Malicious	Semi-honest	Malicious	Semi-honest	Semi-honest	Semi-honest
Party Setting	$n \geq 2$	$n \geq 2$	$n \geq 2$	$n \geq 2$	2	2	2	2
Comm[a](NLO[b])	$O(n)$	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Comp[a](NLO)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Comm (SC[b])	$O(n)$	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Comp (SC)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$

[a] "Comm" = Communication, "Comp" = Computation.
[b] "NLO" = Nonlinear Operations, "SC" = Share Conversions.

ABY3 [42] introduced three-party malicious security with optimized nonlinearities; and CrypTFlow2 [51] combined secret sharing, HE, and function secret sharing for efficient DNN inference. General-purpose frameworks such as MP-SPDZ, MOTION, and Silph [6, 11, 36] provide modular multi-party support with flexible cryptographic combinations. Recent advances address large-model inference and circuit-level security, as seen in GForce, Bolt, and SecureTLM [12, 45, 48], alongside foundational improvements in maliciously secure arithmetic-Boolean primitives [16, 24]. Table 1 highlights some state-of-the-art protocols and their three key limitations:

Efficient Nonlinear Computation. Maliciously secure protocols incur substantial overhead for nonlinear operations (e.g., comparisons, activations) [42], typically relying on garbled circuits or polynomial approximations over arithmetic shares [7,11,16,24,36,51,56]. In multi-party settings, communication complexity is generally $O(n^2)$, and approximation-based methods may degrade accuracy [11, 16]. Most efficient protocols target two or three parties; multi-party protocols require at least quadratic communication [6,11,16,24,36] due to pairwise verification. Scalable protocols exist only for the semi-honest setting [3, 27], and malicious security typically incurs quadratic overhead [5, 18, 26].

Share Conversion Protocols. Hybrid MPC relies on conversions between arithmetic and Boolean sharings, which remain expensive in the malicious multi-party setting. Protocols such as daBit [53] and edaBit [24] provide authenticated conversions but incur $O(n^2)$ to $O(n^3)$ communication. Even with optimizations like silent OT or threshold FHE, overhead remains significant. Other frameworks [11, 16, 36] employ similar conversion techniques, resulting in synchronization and communication bottlenecks as the number of parties grows.

Security-Performance Trade-off. Recent frameworks have expanded the frontier of privacy-preserving inference through diverse designs. BumbleBee [40] and Bolt [48] achieve efficient Transformer inference under the semi-honest model but lack malicious robustness and scalability. SHAFT [35] and SHARK [31] leverage function secret sharing (FSS) with correlated randomness for two-party inference setting, offering fast online performance but posing challenges for extension to multi-party settings due to correlated key generation. MOTION [6] and Garg et al. [27] provide scalable hybrid MPC with semi-honest security, while MD-ML [59] and MD-SONIC [61] achieve malicious security in the dishonest majority model, where nonlinear operations require $O(\log_2 k)$ online rounds in the bit-length k. Together, these developments reveal a persistent tension: existing protocols balance efficiency, scalability, and adversarial strength differently, leaving a gap between theoretical soundness and practical deployment for multi-party PP-LLM.

1.1 Our Results

We present **SMASH**, a novel maliciously secure hybrid MPC framework that leverages arithmetic secret sharing and LUT to achieve constant computational and communication overhead for nonlinear function evaluation, independent of function complexity. These advances are not merely optimizations but constitute new building blocks that enable secure hybrid MPC for LLMs, as summarized in Table 1.

Core Ingredients of SMASH. SMASH constructs a maliciously secure pipeline strictly between Arithmetic and SM-LUT shares, built upon two key techniques:

(i) Scalable Maliciously Secure Lookup Table (SM-LUT). SM-LUT (Section 5) enables efficient nonlinear operations by accepting arithmetic-shared indices and outputting table values as arithmetic shares. Its rotation-based blinding mechanism, utilizing the discrete Fourier transform (DFT), ensures efficiency, while malicious security is enforced via ZKPoKs (Section 4). Crucially, the offline phase incurs $O(n)$ communication and rounds, while the online phase achieves constant communication and rounds (independent of bit-length and n), making SMASH inherently scalable and WAN-friendly.

(ii) Efficient Share Conversion between LUT and Arithmetic Shares (L2A/A2L). We introduce lightweight protocols for converting between arithmetic-sharing-based MPC and SM-LUT (Section 6), relying solely on addition and share reconstruction. This enables seamless integration of linear and nonlinear computation with minimal overhead.

Optimized Boolean-Arithmetic Share Conversions (A2B/B2A). We propose optimized protocols for semi-honest secure Boolean-arithmetic share conversions leveraging the semi-honest variant of SM-LUT, prioritizing ultra-low latency in mixed-protocol settings (Section 6).

Comprehensive Performance Evaluation. We implement SMASH and its semi-honest version (without ZKPoKs) and evaluate them against recent maliciously secure frameworks (MD-ML, MD-SONIC, SHARK) and semi-honest systems (Bolt, BumbleBee, SHAFT). Our evaluation covers nonlinear function evaluation, share conversion, and end-to-end secure inference. For nonlinear operations, SMASH achieves low runtime and communication alongside high scalability, and maintains consistently lower numerical error than other frameworks. In share conversion, A2L/L2A reduce communication and runtime by over $10^3\times$, while A2B/B2A reduce offline runtime by up to two orders of magnitude. For end-to-end Transformer inference, SMASH reduces cumulative nonlinear runtime by up to $18.9\times$ and communication by over $10^3\times$, effectively removing the nonlinear bottleneck, and achieves state-of-the-art efficiency in semi-honest settings. SM-LUT on Qwen3-0.6B demonstrates linear scaling with token count and preserves high inference fidelity, with only 2–5% accuracy loss relative to plaintext execution.

2 Preliminaries

We use the standard ideal/real paradigm [8] to prove the security of our protocols against malicious and static adversaries. Below we recall the preliminaries required throughout the paper; extended preliminaries are deferred to Appendix A.

Notation. Let $\mathcal{P} = \{P_1, \ldots, P_n\}$ denote the set of n parties. PPT refers to probabilistic polynomial-time algorithms. We write $\langle x \rangle^a$ and $\langle x \rangle^b$ for the arithmetic and Boolean secret shares of x, respectively, with $\langle x \rangle_i^a$ and $\langle x \rangle_i^b$ denoting the shares held by party P_i.

ElGamal Encryption. We adopt lifted ElGamal encryption [23] over a cyclic group $\mathbb{G}$ of prime order q, where the plaintext space is $\mathcal{M} \subset \mathbb{Z}_q$. It involves four algorithms: Setup to generate public parameters, key generation Gen where $sk \in \mathbb{Z}q$ yields $pk = g^{sk}$ (denoted by h in ZKPoK for simplicity), randomized encryption $\mathsf{Enc}_{pk}(m) = (g^r, g^m h^r) = c$ using randomness r, and decryption $\mathsf{Dec}_{sk}(c) = \log_g(b/a^{sk}) = m$. It is chosen-plaintext attack (CPA)-secure and supports additive homomorphism: $\mathsf{Enc}(m_1; r_1) \cdot \mathsf{Enc}(m_2; r_2) = \mathsf{Enc}(m_1 + m_2; r_1 + r_2), \alpha \cdot \mathsf{Enc}(m; r) = \mathsf{Enc}(\alpha m; \alpha r)$.

Discrete Fourier Transform. The DFT over $\mathbb{Z}_q$ uses a primitive $\bar{n}$-th root of unity $\alpha \in \mathbb{Z}_q$. Given input $\{x_k\}_{k=0}^{\bar{n}-1}$, DFT and IDFT are defined as: $x'_k = \sum_{j=0}^{\bar{n}-1} x_j \alpha^{kj} \bmod q$ and $x_k = 1/\bar{n} \sum_{j=0}^{\bar{n}-1} x'_j \alpha^{-jk} \bmod q$. Due to the homomorphic property of ElGamal encryption, DFT can be computed directly on encrypted values: $\{\mathsf{Enc}(x'_k)\}_{k=0}^{\bar{n}-1} \leftarrow \mathsf{DFT}(\bar{n}, \alpha, \{\mathsf{Enc}(x_k)\}_{k=0}^{\bar{n}-1})$, $\{\mathsf{Enc}(x_k)\}_{k=0}^{\bar{n}-1} \leftarrow \mathsf{IDFT}(\bar{n}, \alpha, \{\mathsf{Enc}(x'_k)\}_{k=0}^{\bar{n}-1})$.

Pedersen Commitment. Pedersen commitment [49] consists of three algorithms: KGen outputs public parameters $(g, \tilde{h})$ over a group $\mathbb{G}$ of prime order q; $\mathsf{Com}(m, r)$ computes a commitment $c = g^m \tilde{h}^r$ to a message $m \in \mathbb{Z}_q$ using randomness r; and $\mathsf{Ver}(c, (m, r))$ verifies that c opens correctly to m. For formal definitions, please refer to [49].

Threshold Homomorphic Encryption. We employ an $(n-1, n)$-threshold homomorphic encryption (THE) scheme over message space $\mathcal{M}$, where the secret key sk is additively shared among n parties so that any coalition of $n-1$ parties learns nothing about sk. Following [27], a THE scheme consists of the following algorithms: $\mathsf{Setup}(1^\lambda)$ generates public parameters pp; $\mathsf{SecKeyGen}(pp)$ lets each party P_i sample a share sk_i, with global secret key $sk = \sum_{i=1}^n sk_i$; $\mathsf{PubKeyGen}(pp)$ is an interactive protocol among parties to derive the public key $pk = \mathsf{PubKeyGen}(sk_1, \ldots, sk_n)$; encryption is given by $\mathsf{TEnc}_{pk}(m)$ which outputs a ciphertext $[\![m]\!]$ for $m \in \mathcal{M}$; and decryption is realized through $\mathsf{TDec}_{sk}([\![m]\!])$, where parties jointly recover m from $(sk_1, \ldots, sk_n)$ and $[\![m]\!]$. The scheme is required to satisfy standard correctness and CPA security, and any coalition of fewer than n parties cannot derive information about sk. Security proofs typically follow the simulation-based paradigm [8].

Σ-Protocol. A Σ-protocol is a three-move public-coin proof for non-deterministic polynomial relations (x, w) [50]. It satisfies *completeness*, *special soundness* and *honest-verifier zero-knowledge (HVZK)*. For formal definition, please refer to [50]. We apply Fiat-Shamir [25] to obtain non-interactive proofs, written as $\pi(x; w)$.

Arithmetic Black Box and Conversions. Following [27, 37], we model secure computation via an ABB functionality $\mathcal{F}_{\text{ABB}}$, which supports Input, Random, Encrypt, Linear Combination, Multiply, and Output. Our protocol extends this functionality with conversions between arithmetic, Boolean, LUT, and encrypted shares. We defer the full syntax of the conversion box $\mathcal{F}_{\text{Con}}$ to Figure 13 in Appendix A.

3 Technical Overview

This section provides the technical overview of SMASH. We first review the existing MPC-based approaches for nonlinear activation evaluation and share conversion for PP-LLM, highlighting their limitations in scalability, malicious security, and efficiency. We then present our SMASH framework, which integrates SM-LUT with lightweight share conversion protocols to enable scalable, maliciously secure hybrid MPC.

3.1 Previous Solutions for PP-LLM

The State-of-the-art Frameworks. Recent advances in privacy-preserving inference have explored multiple design paradigms for improving both efficiency and robustness. BumbleBee [40] and Bolt [48] propose Transformer-oriented arithmetic optimizations and layer fusion techniques, achieving excellent throughput in semi-honest settings, but these protocols lack malicious security. Semi-honest secure SHAFT [35] and maliciously secure SHARK [31] adopt FSS with correlated randomness to achieve two-party inference. These designs offer near-constant online latency and provable

correctness, yet rely on synchronized key generation and correlation between two parties, complicating extension to $n > 2$ parties. In contrast, MD-ML [59] and MD-SONIC [61] realize full malicious security for the dishonest-majority case through pairwise authenticated multiplications and multi-round verification, where nonlinear operations such as ReLU require $O(\log_2 k)$ online rounds in the bit-length k. While these protocols ensure strong security guarantees, they incur either $O(n^2)$ cost or remain sensitive to WAN latency, limiting scalability for large distributed inference.

Techniques for Nonlinear Evaluation in PP-LLM. Most MPC approaches for nonlinear functions employ polynomial approximations to simulate activations such as ReLU and GELU [31, 38, 59, 61]. However, these are typically limited to two-party, require multi-round, or suffer from accuracy loss (e.g., over 5% for BERT-base in MPCFormer [38]). MP-SPDZ supports n-party malicious security, using polynomial approximations for complex functions [36], but incurs $O(n^2)$ overhead and still experiences accuracy loss.

Lookup Table (LUT) efficiently evaluates exact nonlinear functions but is mostly restricted to two-party settings using Garbled Circuits (GC) and correlated randomness [7, 17, 19, 32, 33, 54]. These frameworks scale poorly to n parties because GC-based primitives (e.g., BMR [2]) may incur quadratic complexity. Garg et al. [27] proposed a scalable semi-honest LUT protocol leveraging BGV encryption. In their approach, parties sequentially re-randomize and permute an entrywise-encrypted table using private masks $\{r_i\}$, effectively blinded by a global XOR-share $r = \bigoplus r_i$. To evaluate the LUT, parties reconstruct the masked index $j \oplus r$ to retrieve the corresponding ciphertext, which is then converted into an arithmetic share. While maintaining noise via bootstrapping, the protocol achieves linear communication complexity. We adopt this construction as our foundation, as its linear scalability offers a more viable path to malicious security than existing n-party extensions of GC-based methods.

However, extending it to malicious security setting introduces substantial challenges:

(i) Zero-Knowledge Proof Approaches. While ZKP techniques (e.g., Σ-protocols, Groth16, Plonk [26, 30, 60]) enable input-oblivious verification, their application to homomorphic LUTs incurs significant overhead. Specifically, proving the correctness of encrypted table lookups requires demonstrating encrypted permutations, resulting in complexity $O(M \log s \log c)$, where M is the table size, s the slot size, and c the ciphertext space [34].

(ii) MAC-Based Approaches. SPDZ-style MACs necessitate circuit representations of lookup operations, hence incurring similar complexity. Furthermore, MAC generation scales quadratically with the number of parties ($O(n^2)$), making such approaches impractical for large-scale settings.

Techniques for Share Conversion. Escudero et al. [24] introduced edaBit, a protocol for maliciously secure share conversion in hybrid MPC. The protocol generates random $r \in \mathbb{Z}_q$ and shares it both arithmetically ($\langle r \rangle^a$) and bitwise ($\langle r_0 \rangle^b, \ldots, \langle r_{\ell-1} \rangle^b$). Correctness is enforced via a cut-and-choose approach: parties generate multiple edaBits, randomly open a subset for verification, and then shuffle and pair the remaining ones to check sum consistency across both domains. Arithmetic shares use additive sharing, while bitwise shares require secure ℓ-bit Boolean addition, incurring $O(n\ell)$ AND gates. Maliciously secure AND triples, produced via OT extension or threshold FHE, result in $O(n^3\ell)$ or $O(n^2\ell)$ communication costs, respectively.

Summary. Existing MPC-based PP-LLM schemes face significant challenges: *(i)* nonlinear operations incur high costs under malicious security, *(ii)* scalability is hindered by $O(n^2)$ overhead, *(iii)* share conversion protocols such as edaBit are inefficient in the multi-party setting, and *(iv)* achieving both strong security and practical efficiency remains elusive. To address these limitations, we introduce SMASH, a scalable, maliciously secure framework that enables efficient nonlinear evaluation via SM-LUT and lightweight share conversions. Our protocols achieve substantial improvements over prior works while preserving security and accuracy.

3.2 Overview of SMASH

As depicted in Figure 1, we present a novel hybrid MPC framework for PP-LLM that enables efficient nonlinear computation and integration with arithmetic-sharing-based protocols. The central component, SM-LUT, is a scalable lookup protocol that evaluates nonlinear functions by performing masked table rotations in the DFT domain on arithmetic-shared indices, and correctness is efficiently ensured via ZKPoKs. While the interactive structure of SM-LUT follows the high-level idea of masked lookup from Garg et al. [27], our construction departs significantly in technique, particularly in its use of arithmetic-domain rotation and adversarial verifiability. To facilitate hybrid computation, we introduce lightweight share conversion protocols that bridge SM-LUT and standard SPDZ-style arithmetic MPC. Together, these components provide efficient, scalable, and verifiable under malicious security.

Threat Model. We consider a dishonest-majority setting with malicious adversaries. In our framework, any subset of parties may arbitrarily deviate from the prescribed protocol, collude, or attempt to infer private information from intermediate messages. The security goal of SMASH is to ensure that no information about the private inputs or model parameters can be inferred from the transcript and outputs. In particular, both the input data and the model parameters remain confidential against all parties, even in the presence of colluding active adversarial behavior.

Overview of SM-LUT. As depicted in Figure 1, SM-LUT comprises two main sub-protocols:

(i) Offline Preprocessing. Parties first compute $\mathbf{c}' = \mathsf{DFT}(\mathbf{c})$, where $\mathbf{c}$ is the ciphertext vector of the table T. A jointly

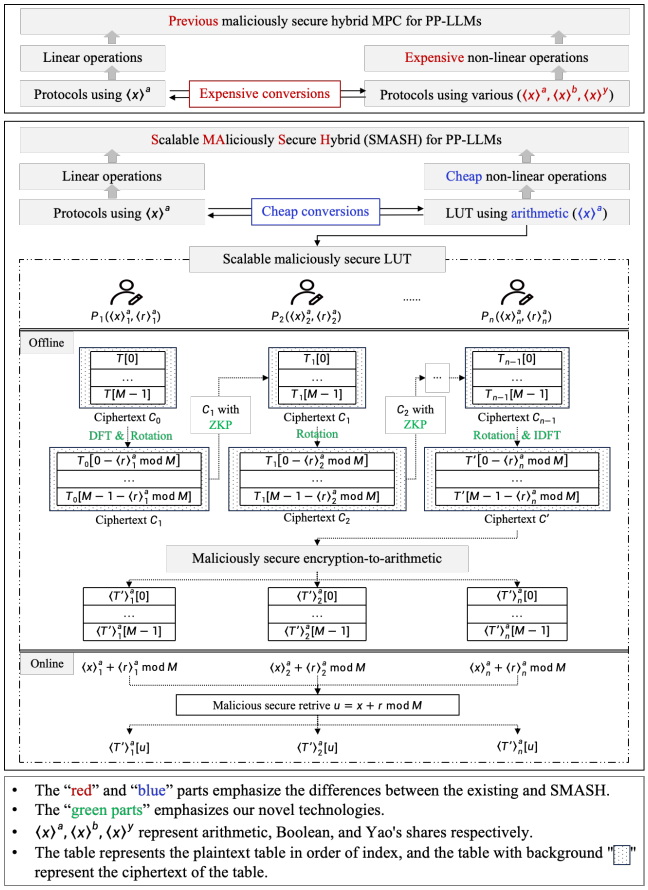

Figure 1: The frameworks of the existing hybrid protocols and our hybrid protocol based on SM-LUT.

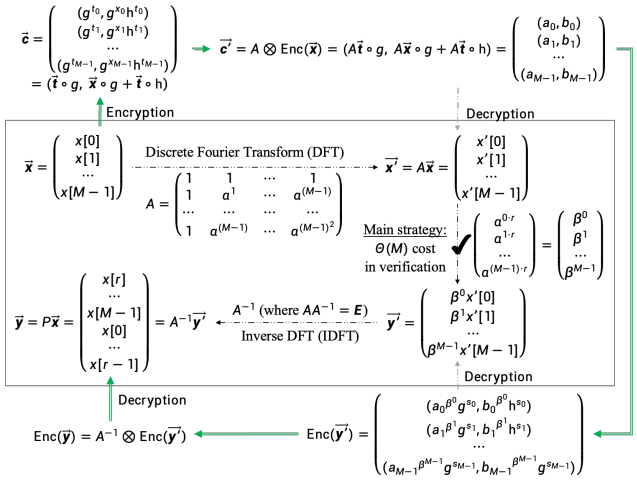

Figure 2: Applying DFT to rotation for encrypted-table

sampled random shift r is then applied in the DFT domain via multiplicative masking, with correctness verified through ZKPoKs that preserve the privacy of both the lookup index and the shift. Subsequently, an encryption-to-arithmetic conversion protocol is invoked to obtain additive shares of the masked table $T' = \mathsf{IDFT}(T)$, where $T'[i] = T[i - r \bmod M]$ for $i \in [0, M-1]$.

(ii) Constant-round Online Lookup. Each party locally computes the masked index $\langle u \rangle^a$ by adding the random shift $\langle r \rangle^a$ to the input index $\langle x \rangle^a$, and then retrieves the corresponding entry from its share of the masked table.

Verification Design for Rotation. To ensure malicious security, we must verify the correctness of the rotation applied to the encrypted table. We first give a straightforward idea and then present our main strategy.

(i) Naive Approach: Reducing Permutation Complexity. A direct method to reduce the cost of proving permutation correctness is to minimize the computational complexity of the permutation itself. Given an encrypted table $T = (c[0], c[1], \ldots, c[M-1])$, a rotation by r yields $(c[r], c[r+1], \ldots, c[M-1], c[0], \ldots, c[r-1])$. With arithmetic secret sharing, this corresponds to a left-rotation via a permutation matrix. When M is a power of two, this operation can be accelerated using the Number Theoretic Transform (NTT) [34], reducing the computational cost from $O(M \log s \log c)$ to $O(M \log M)$.

However, while this lowers the cost of the permutation itself, it does not naturally integrate with Σ-protocols. Generating proofs using Groth [30] or Plonk [26] incurs an overall cost of $O(nM \log M)$, which remains impractical.

(ii) Our Approach: DFT-Based Efficient Verification. Inspired by de Hoogh et al. [21], we propose an optimized verification method based on the Discrete Fourier Transform (DFT). As shown in Figure 2, we apply the DFT over $\mathbb{Z}_q$ using a public Vandermonde matrix, selecting a Type III elliptic curve in $\mathbb{G}_1$ following [20] to ensure efficiency under the DDH assumption. By transforming the permutation problem into the frequency domain, we reduce the secure shuffle to a series of secure exponentiations. This reformulation allows us to employ a lightweight ZKPoK on ElGamal-encrypted tables, treating the shared secrets as the witness. Compared to the foundational construction by de Hoogh et al., our design significantly reduces the communication and computation overhead by 50%. Special soundness is guaranteed via the multiple forking lemma, which enables witness extraction through iterative rewinding [13]. This approach enables a lightweight ZKPoK with complexity $O(M)$, resulting in total complexity $O(Mn)$, which is lower than generic constructions. Furthermore, this approach introduces no approximation error, offering clear advantages over existing library-based methods.

Overview of Share Conversion Protocols. We present four protocols for converting between arithmetic, LUT, and Boolean shares, which rely threshold ElGamal encryption and MAC-based consistency checks to ensure correctness.

(i) Arithmetic and LUT: The maliciously secure A2L and L2A convert between arithmetic and LUT shares by combining threshold homomorphic encryption with MAC-based arithmetic sharing. Parties encrypt their local shares and ran-

dom masks under a threshold encryption scheme and prove consistency with the MAC generation process via ZKPoKs. A masked value is opened using arithmetic MPC and independently recovered by threshold decryption, and consistency is enforced by checking equality of the two results.

(ii) Arithmetic and Boolean: The semi-honest secure A2B and B2A convert between arithmetic and Boolean shares, following the LUT-based design paradigm of Garg et al. [27]. B2A converts Boolean shares to arithmetic shares by mapping each bit through a small LUT and reconstructing the value via linear combination. A2B masks the arithmetic value with a random Boolean share, opens the masked difference, and recovers Boolean shares via a LUT-based modular addition.

4 Building Blocks for SM-LUT

We develop three core primitives for SM-LUT. We design a suite of ZKPoKs, construct a threshold homomorphic ElGamal scheme supporting maliciously secure decryption, and present a conversion protocol from encryption to LUT shares.

4.1 Our Zero-Knowledge Proofs of Knowledge

We describe an efficient Σ-protocol for verifying the correctness of ciphertext rotation, followed by additional proofs required within SM-LUT.

Bandwidth-Efficient Σ-Protocol for Ciphertext Rotation. We develop a communication-efficient ZKPoK for ElGamal ciphertext rotation in Figure 3, inspired by de Hoogh et al.'work [21].

Theorem 1. *The protocol is secure under the discrete logarithm (DL) assumption in the group $\mathbb{G}$ in the RO model.*

The proof of Theorem 1 is provided in Appendix C.1. We use $\Pi_{\text{Rot}}(g,h,\tilde{h},\{a_k,b_k,d_k,e_k\}_{k=0}^{\bar{n}-1};\beta,\{s_k\}_{k=0}^{\bar{n}-1})$ to denote the proof that proves the following relation: $\{g,h,\tilde{h},\{a_k,b_k,d_k,e_k\}_{k=0}^{\bar{n}-1};\beta,\{s_k\}_{k=0}^{\bar{n}-1} | d_k = a_k^{\beta^k} g^{s_k},\ e_k = b_k^{\beta^k} h^{s_k}, \forall k \in [0,\bar{n}-1]\}$.

Additional ZKPoKs. In addition to Π_{Rot}, our protocol employs the following ZKPoKs to prove the corresponding relations:

- $\Pi_{\text{Log}}(g,y;x)$ employs the Schnorr protocol of discrete logarithm [55] for the relation $\mathsf{R}_{\text{Log}} = \{(g,y;x) \mid y = g^x\}$.
- $\Pi_{\text{Exp}}(g,g_1,y_1,y_2;x)$ employs the proof of exponentiation from Chow et al. [13] for $\mathsf{R}_{\text{Exp}} = \{(g,g_1,y_1,y_2;x) \mid y_1 = g^x, y_2 = g_1^x\}$.
- $\Pi_{\text{Dec}}(g,h,x,y_1,y_2;r)$ using $\Pi_{\text{Exp}}(g,h,y_1,y_2/g^x;r)$ for $\mathsf{R}_{\text{Dec}} = \{g,h,x,y_1,y_2;r | y_1 = g^r, y_2 = g^x h^r\}$.
- $\Pi_{\text{Enc}}(g,h,y_1,y_2;x,r)$ for $\mathsf{R}_{\text{Enc}} = \{(g,h,y_1,y_2;x,r) \mid y_1 = g^r, y_2 = g^x h^r\}$.
- $\Pi_{\text{Ran}}(p,g,g_1,h,y_1,y_2,y_3;x,r)$ for $\mathsf{R}_{\text{Ran}} = \{(p,g,g_1,h, y_1,y_2,y_3;x,r) \mid y_1 = g^r, y_2 = g^x g_1^r, y_3 = g^x h^r, x \in [0,p)\}$.

Interact ZKPoK Π_{Rot}

Input: The Prover knows (β, s_k) such that $\beta \in \langle\alpha\rangle$ and $(d_k,e_k) = Y'_k = (a_k^{\beta^k} g^{s_k}, b_k^{\beta^k} h^{s_k})$ for $k \in [0,\bar{n}-1]$. The Verifier knows $g,h,\widetilde{h},(a_k,b_k) = X'_k$, and $(d_k,e_k) = Y'_k$ for $k \in [0,\bar{n}-1]$.

Commitment (Prover): Sample $\widetilde{m},b \leftarrow_R \mathbb{Z}_q$, compute $\widetilde{C} = \widetilde{h}^{\widetilde{m}}$, and set $c_0 = g$ and compute $z_i = H(i,\{a_j,b_j,d_j,e_j\})$ for $i \in [1,3]$]. For $k \in [0,\bar{n}-1]$, sample $m_k,t_k,u_k,v_k,\widetilde{m}_k \leftarrow_R \mathbb{Z}_q$, compute $y_k = H(\{a_j,b_j,d_j,e_j\},k)$, $c_{k+1} = c_k^{\beta}\widetilde{h}^{t_k}$, $C_{k+1} = c_k^{b}\widetilde{h}^{m_k}$, $\widetilde{C}_k = g^{u_k}\widetilde{h}^{\widetilde{m}_k}$, $D_k = a_k^{u_k} g^{v_k}$, $E_k = b_k^{u_k} h^{v_k}$ and $M_k = \widetilde{C}_k^{z_1} D_k^{z_2} E_k^{z_3}$. Compute $C = \prod_{k=0}^{\bar{n}-1} C_{k+1}^{y_k}$ and send $(\widetilde{C},\{c_{k+1},M_k\}_{k=0}^{\bar{n}-1},C)$ to the Verifier.

Challenge (Verifier): Sample $\lambda \leftarrow_R \mathbb{Z}_q$, send it to the Prover.

Response (Prover): Set $t^*_{-1} = 0$ and compute $\sigma = b + \lambda\beta$. For $k \in [0,\bar{n}-1]$, compute $t^*_k = t^*_{k-1}\beta + t_k$, $\psi_k = m_k + \lambda t_k$, $\mu_k = u_k + \lambda\beta^k$, $\nu_k = v_k + \lambda s_k$, $\rho_k = \widetilde{m}_k + \lambda t^*_{k-1}$. Compute $\eta = \widetilde{m} + \lambda t^*_{\bar{n}-1}$ and $\psi = \sum_{k=0}^{\bar{n}-1} \psi_k y_k$. Send $(\sigma,\eta,\psi,\{\mu_k,\nu_k,\rho_k\}_{k=0}^{\bar{n}-1})$ to the Verifier.

Verification (Verifier): Compute $z_i = H(i,\{a_j,b_j,d_j,e_j\})$ for $i \in [1,3]$ and $y_k = H(\{a_j,b_j,d_j,e_j\},k)$ for $k \in [0,\bar{n}-1]$. Check $\widetilde{h}^{\eta} \stackrel{?}{=} \widetilde{C}(c_{\bar{n}}/g)^{\lambda}$ and $(\prod c_k^{y_k})^{\sigma}\widetilde{h}^{\psi} \stackrel{?}{=} C(\prod c_{k+1}^{y_k})^{\lambda}$. For $k \in [0,\bar{n}-1]$, check $g^{z_1\mu_k + z_2\nu_k}\widetilde{h}^{z_1\rho_k} a_k^{z_2\mu_k} b_k^{z_3\mu_k} h^{z_3\nu_k} \stackrel{?}{=} M_k c_k^{z_1\lambda} d_k^{z_2\lambda} e_k^{z_3\lambda}$.

Figure 3: Low-bandwidth proof of a rotation in the transformed domain of ElGamal encryptions

- $\Pi_{\text{Cip}}(g,h,g_1,y_1,y_2,y_3;x,r)$ for $\mathsf{R}_{\text{Cip}} = \{(g,h,g_1,y_1, y_2,y_3;x,r) \mid y_1 = g^r, y_2 = g^x h^r, y_3 = g_1^x h^r\}$.

These relations exhibit additive homomorphism, making them well-suited for Schnorr-style proofs [55]. We defer the detailed protocols of Π_{Enc}, Π_{Ran} and Π_{Cip} to Appendix B. Their completeness, special soundness, and HVZK properties follow directly from the standard Schnorr-style Σ-protocols.

4.2 ElGamal Threshold Homomorphic Encryption

We leverage the ZKPoKs from Section 4.1 to construct a maliciously secure ElGamal-THE tailored for SM-LUT.

Syntax. An ElGamal-THE scheme consists of the following algorithms and protocols:

- $\mathsf{Setup}(1^\lambda)$: On input a security parameter λ, output public parameters $pp = (q,\mathcal{M},\bar{\mathcal{M}},\mathbb{G},\bar{n},\alpha)$, where q is a prime order of group $\mathbb{G}$, $(\alpha,\bar{n}) \leftarrow \mathsf{DSetup}(1^\lambda,q)$, $\bar{\mathcal{M}}$ is the domain for the discrete logarithm problem, and $\mathcal{M} \subset \bar{\mathcal{M}}$ is the plaintext

space.

• $\mathsf{SecKeyGen}(pp)$: Each party P_i samples $sk_i \leftarrow \mathsf{Gen}(pp)$. The global secret key is set $sk = \sum_{i=1}^{n} sk_i \in \mathbb{Z}_q$.

• $\mathsf{PubKeyGen}(pp)$: Each P_i computes $pk_i = g^{sk_i}$ and broadcasts commitment $(C_i, D_i) \leftarrow \mathsf{Com}(pk_i)$. Parties provide $\Pi_{\mathrm{Log}}(g, pk_i; sk_i)$ to prove correctness of opening D_i. The public key is set to $pk = \prod_{i=1}^{n} pk_i (= g^{sk})$.

• $\mathsf{TEnc}_{pk}(m)$: On input pk and $m \in \mathcal{M}$, output $[\![m]\!] = (g^r, g^m h^r)$ for random $r \leftarrow_R \mathbb{Z}_q$.

• $\mathsf{TDec}_{sk}([\![m]\!])$: Given $[\![m]\!] = (a,b)$, each P_i broadcasts a^{sk_i} with proof $\Pi_{\mathrm{Exp}}(g, a, pk_i, a^{sk_i}; sk_i)$. All parties reconstruct $m = \log_g \left(b / \prod_{i=1}^{n} a^{sk_i}\right)$.

• $\mathsf{Rotate}_{pk}([\![x']\!], r, \bar{n})$: On input $[\![x']\!] = (a,b)$, r and $\bar{n}$, where $x' = \mathsf{DFT}(\alpha, x)$, parse (a,b) as $((a_0, \ldots, a_{\bar{n}-1}), (b_0, \ldots, b_{\bar{n}-1}))$. Compute $\beta = \alpha^r$ and sample $\{s_k\}_{k=0}^{\bar{n}-1} \leftarrow_R \mathbb{Z}_q^{\bar{n}}$. For $k \in [0, \bar{n}-1]$, compute $d_k = a_k^{\beta^k} g^{s_k}$ and $e_k = b_k^{\beta^k} h^{s_k}$. It outputs $[\![y']\!] = (d, e)$.

Linear Combination. This follows directly from the additive homomorphism of lifted ElGamal.

Correctness of Decryption. Correctness is immediate from the underlying ElGamal scheme, as ciphertexts are valid encryptions under the public key and decryption proceeds via the threshold protocol.

Correctness and Verification of Rotation. For a plaintext vector x with DFT x', we have $x'_k = \sum_j x_j \alpha^{kj}$. Multiplying by α^{rk} in the DFT domain yields a circular rotation by r in the time domain by the shift theorem. Thus, the inverse DFT of y' gives the rotated vector $\{x_{(j-r) \bmod \bar{n}}\}_{j=1}^{\bar{n}}$. As long as re-randomization is consistent and $\beta = \alpha^r$ is taken modulo the DFT order, both semantic security and correctness hold. For verifiability, the rotator provides a ZKPoK $\Pi_{\mathrm{Rot}}(g, h, \tilde{h}, \{a_k, b_k, d_k, e_k\}_{k=0}^{\bar{n}-1}; \beta, \{s_k\}_{k=0}^{\bar{n}-1})$, succinctly denoted as $\Pi_{\mathrm{Rot}}(g, h, \tilde{h}, [\![x']\!], [\![y']\!]; \beta, [\![s]\!])$.

Privacy. ElGamal-THE guarantees privacy against any coalition of less than n parties, since the secret key is additively shared. The $\mathsf{PubKeyGen}$ protocol is simulatable as in [9], ensuring no information leakage.

CPA Security. CPA security is inherited from standard ElGamal [23], as the threshold variant preserves the same security properties. Any adversary breaking ElGamal-THE can be reduced to an adversary against ElGamal.

4.3 Conversion from Encryption to LUT Shares

Preparing for SM-LUT, we describe a conversion protocol converting ElGamal-THE ciphertexts into LUT shares. This protocol follows the construction in [44], with BFV-THE replaced by ElGamal-THE.

Theorem 2. *Protocol Π_{E2L} (Figure 4) securely realizes the encryption-to-LUT share conversion for functionality $\mathcal{F}_{Con}$ (Figure 13) against malicious adversaries in the dishonest majority setting, assuming the discrete logarithm assumption holds and ElGamal-THE is CPA-secure.*

Protocol $\Pi_{\mathbf{E2L}}$

Input: Parties $P_1, \ldots, P_n$ hold the following inputs: The set of public parameters $pp = (q, \mathcal{M}, \bar{\mathcal{M}}, \mathbb{G}, \bar{n}, \alpha)$ for ElGamal-THE, where $\mathcal{M} = \mathbb{Z}_p, \bar{\mathcal{M}} = \mathbb{Z}_{p'}, p' = np$. A ciphertext $[\![x]\!] = (c_0, c_1)$. P_i holds a share sk_i of the secret key sk and $\{pk_j\}_{j=1}^{n}$.

Conversion from encryption to LUT shares:

1. For $i > 1$, P_i chooses $\langle x \rangle_i^a \leftarrow \mathcal{M}$ and broadcasts $l_i := g^{\langle x \rangle_i^a} c_0^{sk_i}$, $cip_i := (pk_i, \tilde{c}_i = g^{\langle x \rangle_i^a} h^{sk_i})$ and $\Pi_{\mathrm{Ran}}(p, g, c_0, h, pk_i, l_i, \tilde{c}_i; \langle x \rangle_i^a, sk_i)$.
2. P_1 computes $l := g^{(n-1)p} c_1 \prod_{i=2}^{n} l_i^{-1}, y := \log_g(c_0^{-sk_1} l), \langle x \rangle_1^a := y \bmod p, l_1 := g^{\langle x \rangle_1^a} c_0^{sk_1}, cip_1 := (pk_1, \tilde{c}_1 = g^{\langle x \rangle_1^a} h^{sk_1})$, and broadcasts $\Pi_{\mathrm{Ran}}(p, g, c_0, h, pk_1, l_1, \tilde{c}_1; \langle x \rangle_1^a, sk_1)$.
3. When passing verifications, parties output $\langle x \rangle^a$, $\{cip_i\}_{i=1}^{n}$.

Figure 4: Protocol for converting ElGamal-THE encryption into LUT shares

The proof of Theorem 2 is provided in Appendix C.2.

5 Scalable Maliciously Secure Lookup Table

With the building blocks described in Section 4, we can now introduce SM-LUT, which supports both public lookup tables and privately shared tables. SM-LUT is divided into two subprotocols: the *offline preprocessing* sub-protocol $\Pi_{\text{Prep-LUT}}$, which generates a masked table, and the *online* sub-protocol Π_{Lookup}, which utilizes the masked table to securely evaluate lookups. We formally define the offline phase of both public and private masked tables through the functionality $\mathcal{F}_{\text{Prep-LUT}}$ based on the definition by Garg et al. [27], as illustrated in Figure 5. Specifically, $\mathcal{F}_{\text{Prep-LUT}}$ samples a random value r and applies a cyclic shift to the table T of size M, generating additive shares of the masked table T' and the shift value r. The masked table is computed as $T'[j] = T[j - r \bmod M], \forall j \in [0, M-1]$. By invoking $\mathcal{F}_{\text{Prep-LUT}}$, Π_{Lookup} securely realizes the functionality $\mathcal{F}_{\text{LUT}}$, as depicted in Figure 6.

5.1 Offline Preprocessing for Masked Tables

We propose a multi-party offline protocol for masking lookup tables, detailed in Figure 5. Protocol $\Pi_{\text{Prep-LUT}}$ securely works in the $\mathcal{F}_{\text{Con}}$-hybrid model and supports both public tables and arithmetic-sharing private tables. It employs a threshold homomorphic encryption (THE) scheme, a commitment scheme, and discrete Fourier transforms. During

Functionality $\mathcal{F}_{\text{Prep-LUT}}$

Let M be the length of a public/private table. This functionality has all of the same features as $\mathcal{F}_{\text{ABB}}$ with the following additional commands.

Public masked table: Upon receiving (MaskedPubTab, T, $\mathbf{c}_T$, id_1, $\mathbf{id}_2$, $\mathbf{id}_3$, id_4) from all parties, where $T \in (\mathbb{Z}_q)^M$, $\mathbf{c}_T = (c_{T[0]}, \ldots, c_{T[M-1]})$ is a vector of ElGamal ciphertexts of T, id_1 is a fresh identifier, and $\mathbf{id}_2$,$\mathbf{id}_3$ are vectors of fresh identifiers with respective length M. Sample $r \leftarrow [0, M-1]$. Store (id_1, arith, r), ($\mathbf{id}_2[j]$, arith, $T[s]$) and ($\mathbf{id}_3[j]$, enc, $c'_{T[s]}$) for $j \in [0, M-1]$, and (id_4, enc, c_r), where $s = j - r \bmod M$, c_r is an ElGamal encryption of r, and $c'_{T[s]}$ is a new ElGamal encryption of $T[s]$.

Private masked table: Upon receiving (MaskedPriTab, $\mathbf{c}_T$, $\mathbf{id}_1$, id_2, $\mathbf{id}_3$, $\mathbf{id}_4$, id_5) from all parties, where $\mathbf{c}_T = (c_{T[0]}, \ldots, c_{T[M-1]})$ is a vector of ElGamal ciphertexts of T, ($\mathbf{id}_1[j]$, arith) for all $j \in [0, M-1]$ are present in memory, id_2 is a fresh identifier, and $\mathbf{id}_3$, $\mathbf{id}_4$ are vectors of fresh identifiers with respective length M. Retrieve ($\mathbf{id}_1[j]$, arith, $T[j]$) for $j \in [0, M-1]$, set the vector T accordingly, and sample $r \leftarrow [0, M-1]$. Store (id_2, arith, r) for $j \in [0, m-1]$, ($\mathbf{id}_3[j]$, arith, $T[s]$) and ($\mathbf{id}_4[j]$, enc, $c'_{T[s]}$) for $j \in [0, M-1]$, and (id_5, enc, c_r), where $s = j - r \bmod M$, c_r is an ElGamal encryption of r, and $c'_{T[s]}$ is a new ElGamal encryption of $T[s]$.

Figure 5: Functionality for masked lookup table

the offline phase, each party provides randomness and commitments to jointly apply a random cyclic rotation on the encrypted table, with ZKPoKs ensuring ciphertext integrity and rotation correctness. Depending on the table type, parties either encrypt the public table or aggregate encrypted shares, apply DFT, and propagate ciphertexts through a sequence of masked rotations. After performing IDFT on the final rotated ciphertexts, the result is converted into additive shares. The protocol outputs these shares and rotation along with corresponding encryptions, ensuring the table is correctly masked and aligned for secure evaluation.

Theorem 3. *If all n parties $P_1, \ldots, P_n$ execute the protocol $\Pi_{\text{Prep-LUT}}$ faithfully with valid inputs, then the protocol terminates successfully, and all parties obtain correct $\langle T' \rangle^a$, $\langle r \rangle^a$ and the related ciphertexts.*

Theorem 4. *Protocol $\Pi_{\text{Prep-LUT}}$ (Figure 7) securely realizes the functionality $\mathcal{F}_{\text{Prep-LUT}}$ (Figure 5) against malicious adversaries in the dishonest majority setting in the $\mathcal{F}_{\text{Con}}$-hybrid model, assuming the underlying THE scheme is CPA-secure.*

The proofs of Theorems 3 and 4 are provided in Appendix C.3,C.4, respectively.

Functionality $\mathcal{F}_{\text{LUT}}$

Let M be the length of a public/private table. This functionality has all of the same features as $\mathcal{F}_{\text{ABB}}$ with the following additional commands.

Public table lookup: Upon receiving (PubTabLookup, T, $\mathbf{c}_T$, id_1, id_2, id_3) from all parties, where $T \in (\mathbb{Z}_q)^M$, $\mathbf{c}_T = (c_{T[0]}, \ldots, c_{T[M-1]})$ is a vector of ElGamal ciphertexts of T, and (id_1, arith) are present in memory. Retrieve (id_1, arith, x), store (id_2, arith, $T[x]$) and (id_3, enc, $c'_{T[x]}$), where $c'_{T[x]}$ is a new ElGamal encryption of $T[x]$.

Private table lookup: Upon receiving (PriTabLookup, $\mathbf{c}_T$, $\mathbf{id}_1$, id_2, $\mathbf{id}_3$, id_4) from all parties, where $T \in (\mathbb{Z}_q)^M$, $\mathbf{c}_T = (c_{T[0]}, \ldots, c_{T[M-1]})$ is a vector of ElGamal ciphertexts of T, ($\mathbf{id}_1[j]$, arith) for all $j \in [0, M-1]$ and (id_2, arith) are present in memory. Retrieve ($\mathbf{id}_1[j]$, arith, $T[j]$) for $j \in [0, M-1]$ and (id_2, arith, x), set $T \in (\mathbb{Z}_q)^M$ and $x \in [0, M-1]$ accordingly, and store ($\mathbf{id}_3$, arith, $T[x]$) and (id_4, enc, $c'_{T[x]}$), where $c'_{T[x]}$ is a new ElGamal encryption of $T[x]$.

Figure 6: Functionality for lookup table

Communication and Storage Overhead. Parties broadcast masked ciphertext vectors of length M (each with $O(M)$ lifted ElGamal ciphertexts) along with a proof, resulting in $O(Mn(|\mathbb{G}| + \log_2 q))$ bits of total communication. Each party stores additive shares of the masked table and all ciphertexts, for a storage overhead of $O(Mn|\mathbb{G}| + M \log_2 q)$ bits.

5.2 Online Protocol for Lookup Table

The online lookup protocol follows [27], extended for *malicious security*, and operates in the $\mathcal{F}_{\text{Prep-LUT}}$-hybrid model (Figure 8). Given an arithmetic share $\langle x \rangle^a$ of index $x \in M$, it outputs $\langle T[x] \rangle^a$. Specifically, Π_{Lookup} uses a masked, randomly rotated table T' and offset $\langle r \rangle^a$ from $\mathcal{F}_{\text{Prep-LUT}}$. Parties locally add shares of x and r, encrypt, and jointly decrypt to obtain $u = x + r \bmod M$. Consistency is checked against local shares, after which parties output $\langle T'[u] \rangle^a = \langle T[x] \rangle^a$.

Theorem 5. *If all n parties $P_1, \ldots, P_n$ honestly execute protocol Π_{Lookup} with consistent inputs in the $\mathcal{F}_{\text{Prep-LUT}}$-hybrid model, then the protocol terminates successfully and returns correct $\langle T[x] \rangle^a$ and the related ciphertexts.*

Theorem 6. *Protocol Π_{Lookup} (Figure 8) securely realizes functionality $\mathcal{F}_{\text{LUT}}$ against malicious adversaries in the dishonest majority setting in the $\mathcal{F}_{\text{Prep-LUT}}$-hybrid model.*

The proof of Theorem 5 and 6 are given in Appendix C.5 and C.6, respectively.

Communication and Storage Overhead. Parties jointly decrypt the index ciphertext by broadcasting exponentiated group elements with proofs, resulting in $O(n(|\mathbb{G}| + \log_2 q))$

Protocol $\Pi_{\text{Prep-LUT}}$

Input: Parties $P_1,\dots,P_n$ have the following inputs: The THE scheme with TEnc and Rotate. Suppose that the public parameters and public key pk have been established. The commitment scheme with Com that the public parameters have been established. The $\mathsf{DFT}/\mathsf{IDFT}$ with $\bar{n}=M,\alpha$ have been established.

- *Case 1*: Let $T\in(\mathbb{Z}_q)^M$ be a vector corresponding to a size-M public table.
- *Case 2*: Let $\langle T\rangle^a$ be an arithmetic sharing w.r.t. a vector $T\in(\mathbb{Z}_q)^M$ related to a size-M private table. For $i\in[1,n]$, let $\mathbf{c}_i=\mathsf{TEnc}_{pk}(\langle T\rangle_i^a)$ be the ciphertexts of $\{\langle T[j]\rangle_i^a\}_{j=0}^{M-1}$.

Offline preprocessing of a masked table:

1. Each P_i chooses $r_i\leftarrow_R\mathbb{Z}_M, r_i'\leftarrow_R\mathbb{Z}_q$, computes $\beta_i=\alpha^{r_i}, c_{r_i}=(g^{r_i'},g^{r_i}h^{r_i'})$, and broadcasts c_{r_i}.
2. P_1 computes ciphertexts $[\![T]\!]$ by executing the following depending on if T is public.
- In *Case 1*, P_1 runs $[\![T]\!]\leftarrow\mathsf{TEnc}_{pk}(T)$, $[\![T_0]\!]\leftarrow\mathsf{DFT}(\bar{n},\alpha,[\![T]\!])$, and broadcasts $[\![T]\!],\Pi_{\text{Dec}}(g,h,T,[\![T]\!];k)$, where k is the random vector used to encrypt $\tilde{T}$.
- In *Case 2*, P_1 computes $[\![T_0]\!]\leftarrow\mathsf{DFT}(\bar{n},\alpha,\sum_{i=1}^n\mathbf{c}_i)$.
3. When passing all the verifications, from $i=1$ to n, the parties execute the following steps:
a) If $i\neq 1$, P_i receives $[\![T_{i-1}]\!],\Pi_{\text{Cip}}(g,h,\alpha,c_{r_i},\beta_i h^{r_i'};r_i,r_i'),\Pi_{\text{Rot}}(g,h,\tilde{h},[\![T_{i-2}]\!],[\![T_{i-1}]\!];\beta_{i-1},s_{i-1})$ from P_{i-1}.
b) P_i computes $[\![T_i]\!]\leftarrow\mathsf{Rotate}_{pk}([\![T_{i-1}]\!],r_i)$ and computes $\Pi_{\text{Rot}}(g,h,\tilde{h},[\![T_{i-1}]\!],[\![T_i]\!];\beta_i,s_i)$, where s_i is a vector with respective length M.
c) P_i broadcasts $[\![T_i]\!],\Pi_{\text{Cip}}(g,h,\alpha,c_{r_i},\beta_i h^{r_i'};r_i,r_i')$ and $\Pi_{\text{Rot}}(g,h,\tilde{h},[\![T_{i-1}]\!],[\![T_i]\!];\beta_i,s_i)$. Define that T_n is the plaintext of $[\![T_n]\!]$.
4. When passing all the verifications, the parties run $[\![T']\!]\leftarrow\mathsf{IDFT}(\bar{n},\alpha,[\![T_n]\!])$, and call $\mathsf{E2L}$ of $\mathcal{F}_{\text{Con}}$ to convert $[\![T']\!]$ into $\langle T'\rangle^a$ with $[\![\langle T'\rangle_i^a]\!]$ for $i\in[1,n]$, where $T'[j]=T[j-r \bmod M]$ for $j\in[0,M-1]$.
5. The parties output $\langle r\rangle^a,\langle T'\rangle^a,\{c_{r_i},[\![\langle T'\rangle_i^a]\!]\}_{i=1}^n$.

Figure 7: Protocol for the offline preprocessing of masked tables in the $\mathcal{F}_{\text{Con}}$-hybrid model

Protocol Π_{Lookup}

Input: Parties $P_1,\dots,P_n$ have the following inputs: Let M be the table length. The THE scheme with TDec. The commitment scheme with Com that the public parameters have been established. $\langle x\rangle^a$ is the arithmetic shares of a private index $x\in[M-1]$ and $\{c_{x_i}\}_{i=1}^n$ are the corresponding ciphertexts, where c_{x_i} is encrypted by P_i using TEnc.

- *Case 1*: Let $T\in(\mathbb{Z}_p)^M$ be a public vector related to public map f s.t. $T[j]=f(j)$ for $j\in[0,M-1]$.
- *Case 2*: Let $\langle T\rangle^a$ be the arithmetic shares of a private vector T related to a private map.

Offline preprocessing of masked table: In Case 1 (resp., Case 2), all parties call the (MaskedPubTab, T) (resp., (MaskedPriTab)) command of functionality $\mathcal{F}_{\text{Prep-LUT}}$ to generate a masked table $(\langle r\rangle^a,\langle T'\rangle^a)$ with ciphertexts $\{c_{r_i},[\![\langle T'\rangle_i^a]\!]\}_{i=1}^n$, where $r\in[0,M-1]$ and $T'[j]=T[j-r \bmod M]$ for $j\in[0,M-1]$.

Online table lookup: Given $\langle r\rangle^a,\langle T'\rangle^a,\langle x\rangle^a$ and $\{c_{r_i},[\![\langle T'\rangle_i^a]\!],c_{x_i}\}_{i=1}^n$, the parties generate $\langle T[x]\rangle^a$ as follows: All parties compute $c=\sum_{i=1}^n(c_{r_i}+c_{x_i})$ and run $u\leftarrow\mathsf{TDec}_{sk}(c)$. All parties output $\langle T[x]\rangle^a:=\langle T'\rangle^a[u]$ and $\{[\![\langle T'\rangle_i^a[u]]\!]\}_{i=1}^n$.

Figure 8: Online lookup-table protocol

bits of total communication. Each party stores additive shares of the masked table value and all ciphertexts, with a storage overhead of $O(n(|\mathbb{G}|)+\log_2 q))$ bits.

6 Efficient Share Conversion Protocols

To enable integration of SM-LUT into existing MPC-based frameworks, we design efficient conversion protocols that translate between LUT and arithmetic shares (i.e., A2L/L2A). To prioritize high efficiency, we introduce A2B/B2A based on a semi-honest variant of SM-LUT (by removing ZKPs). These are engineered as ideal drop-in replacements for bridging different sharing modalities within performance-critical, hybrid Arithmetic-Boolean MPC infrastructures. We denote arithmetic-sharing-based and Boolean-sharing-based MPC functionality as $\mathcal{F}_{\text{A-MPC}}$ and $\mathcal{F}_{\text{B-MPC}}$ using the black-box MPC model [37]. Arithmetic shares are defined over an abstract ring associated with $\mathcal{F}_{\text{A-MPC}}$. In concrete instantiations such as Overdrive2k [47], the underlying arithmetic may operate over a larger ring to ensure active security, while correctness is defined modulo the effective lower-bit domain; LUT-related operations act on these effective values.

Theorem 7. *Protocol Π_{A2L} (Figure 9) securely realizes A2L in functionality $\mathcal{F}_{Con}$ (Figure 13) against malicious adversaries in the dishonest majority setting in the $\mathcal{F}_{\text{A-MPC}}$-hybrid model, assuming that the THE scheme is CPA secure.*

Theorem 8. *Protocol Π_{L2A} (Figure 10) securely realizes L2A in functionality $\mathcal{F}_{Con}$ (Figure 13) against malicious adversaries in the dishonest majority setting in the $\mathcal{F}_{\text{A-MPC}}$-hybrid model, assuming that the THE scheme is CPA secure.*

Theorem 9. *Protocol Π_{B2A} (Figure 11) securely realizes B2A in functionality $\mathcal{F}_{Con}$ (Figure 13) against semi-honest adversaries in the $\mathcal{F}_{\text{A-MPC}},\mathcal{F}_{LUT}$-hybrid model, assuming that the THE scheme is CPA secure.*

Theorem 10. *Protocol Π_{A2B} (Figure 12) securely realizes A2B in functionality $\mathcal{F}_{Con}$ (Figure 13) against semi-honest*

Protocol Π_{A2L}

Input: Arithmetic-sharing-based MPC functionality $\mathcal{F}_{\text{A-MPC}}$ with LinComb and Open, the THE scheme with TEnc and TDec, and arithmetic shares $\langle x\rangle^{\bar{a}}$ with MACs.

Conversion:

1. Parties generate $\langle r\rangle^{\bar{a}}$ with MACs, where each P_i broadcasts $c_{r_i} = \mathsf{TEnc}(\langle r\rangle_i^{\bar{a}})$ accompanied by a proof binding it to the MAC-generation process via a common challenge. Each P_i broadcasts $c_{x_i} = \mathsf{TEnc}(\langle x\rangle_i^{\bar{a}})$ with a proof of knowledge of $\langle x\rangle_i^{\bar{a}}$.
2. Parties run $u \leftarrow \mathsf{Open}(\langle x+r\rangle^{\bar{a}})$ and run $u' \leftarrow \mathsf{TDec}_{sk}(c)$, where $c = \sum_{i=1}^{n}(c_{r_i} + c_{x_i})$.
3. Parties check that $u = u'$. If not, abort.
4. Parties output $\langle x\rangle^{a}$ with $\{c_{x_i}\}_{i=1}^{n}$.

Figure 9: Protocol for converting arithmetic into LUT share in the $\mathcal{F}_{\text{A-MPC}}$-hybrid model

Protocol Π_{L2A}

Input: Arithmetic-sharing-based MPC functionality $\mathcal{F}_{\text{A-MPC}}$ with LinComb and Open, the TEnc and TDec, and LUT shares $\langle x\rangle^{a}$ and ciphertexts $\{c_{x_i}\}_{i=1}^{n}$.

Conversion:

1. Parties generate $\langle r\rangle^{\bar{a}}$ with MACs and generate $\langle x\rangle^{\bar{a}}$ with MACs using $\langle x\rangle^{a}$, where each P_i broadcasts $c_{r_i} = \mathsf{TEnc}(\langle r\rangle_i^{\bar{a}})$ accompanied by a proof binding it to the MAC-generation process via a common challenge.
2. Parties run $u \leftarrow \mathsf{Open}(\langle x+r\rangle^{\bar{a}})$ and run $u' \leftarrow \mathsf{TDec}_{sk}(c)$, where $c = \sum_{i=1}^{n}(c_{r_i} + c_{x_i})$.
3. Parties check that $u = u'$. If not, abort.
4. Parties output $\langle x\rangle^{\bar{a}}$ with MACs.

Figure 10: Protocol for converting LUT into arithmetic share in the $\mathcal{F}_{\text{A-MPC}}$-hybrid model

adversaries in the $\mathcal{F}_{LUT}, \mathcal{F}_{A\text{-}MPC}$-hybrid model, assuming that the THE is CPA secure.

The proofs of Theorems 7, 8, 9, and 10 are provided in Appendix C.7, C.8, C.9, and C.10, respectively.

7 Evaluation

We evaluate the SMASH framework along four axes: (i) micro-benchmarks for nonlinear operators, (ii) share conversions, (iii) secure BERT inference, and (iv) large-scale applicability to LLMs.

7.1 Experimental settings

Baselines and software. SMASH is implemented in C++17 and builds on `MP-SPDZ` (commit `96aac1e`)

Protocol Π_{B2A}

Input: Functionality $\mathcal{F}_{\text{LUT}}$, Arithmetic-sharing-based MPC functionality $\mathcal{F}_{\text{A-MPC}}$ with LinComb, Boolean shares $\langle x\rangle^{b}$, where $x \in \mathbb{F}_2^l$ and $l = \lceil \log p \rceil$.

Conversion:

1. For $j \in [0, l-1]$, parties call $\mathcal{F}_{\text{LUT}}$ on input $\langle x_j\rangle^{b}$ to obtain $\langle x_j\rangle^{a}, \{c_{x_{j,i}}\}_{i=1}^{n}$. Then they call Π_{A2L} to obtain $\langle x_j\rangle^{\bar{a}}$ for all $j \in [0, l-1]$.
2. Parties compute $\langle x\rangle^{\bar{a}} = \sum_{j=0}^{l-1} 2^j \langle x_j\rangle^{\bar{a}}$ using LinComb.
3. Parties output $\langle x\rangle^{\bar{a}}$.

Figure 11: Protocol for converting Boolean to arithmetic share in the $\mathcal{F}_{\text{A-MPC}}, \mathcal{F}_{\text{LUT}}$-hybrid model

Protocol Π_{A2B}

Input: Functionality $\mathcal{F}_{\text{LUT}}$, Arithmetic-sharing-based MPC functionality $\mathcal{F}_{\text{A-MPC}}$ with LinComb and Open, and $\langle x\rangle^{\bar{a}}$, where $x \in \mathbb{F}_q$ and $l = \lceil \log p \rceil$.

Conversion:

1. Parties generate $\langle r\rangle^{b}$ and call Π_{B2A} on input $\langle r\rangle^{b}$ to obtain $\langle r\rangle^{\bar{a}}$. Parties call LinComb and Open in $\mathcal{F}_{\text{A-MPC}}$ to obtain $u \leftarrow \mathsf{Open}(\langle x-r\rangle^{\bar{a}})$.
2. P_1 sets $\langle u\rangle_1^{b}$ be the bit-decomposition of u. For $j \in [2, n]$, P_i sets $\langle u\rangle_i^{b} = 0$.
3. Parties call the modulo-addition circuit in $\mathcal{F}_{\text{LUT}}$ with inputs $\langle u\rangle^{b}$ and $\langle r\rangle^{b}$ to obtain the $\langle x\rangle^{b}$ corresponding to $x = u + r \bmod p$.
4. Parties output $\langle x\rangle^{b}$ with MACs.

Figure 12: Protocol for converting arithmetic to Boolean share in the $\mathcal{F}_{\text{LUT}}, \mathcal{F}_{\text{A-MPC}}$-hybrid model

and `ScalableMixedModeMPC` (`ee0c415`). We compare against official implementations of prior work: BOLT (`80f165f`), BumbleBee (`47c5560`), SHAFT (`3ceb0ee`), ScalableMixedModeMPC, SHARK (`6957e5e`), MP-SPDZ, MD-ML (`1b0bdbd`), and MD-SONIC (`1729063`). SHAFT, BumbleBee, and MD-SONIC are Python implementations; their communication measurements are directly comparable as protocol-level metrics, while end-to-end runtime comparisons should be interpreted qualitatively due to language and implementation differences.

Models and applicability. Following prior work [31, 35, 40, 48], our primary experiments target the BERT scale. To show broader applicability to modern LLM primitives, we also evaluate SM-LUT on `Qwen3-0.6B` [58] (rotary embeddings, grouped-query attention). This LLM-focused study measures primitive efficiency across representative operator shapes, which constitute the dominant bottlenecks in secure LLM pipelines, rather than evaluating full end-to-end deployments.

Cryptographic parameters. We target 128 bits of computa-

Table 2: Comparison of maliciously secure nonlinear operations (input size = 10,000).

Framework	Function	Party	Local (s)	LAN (s)	WAN (s)	Comm. (MB)
SHARK (2PC)	GeLU	2	0.019	0.018	0.513	0.324
	ReLU		0.013	0.018	0.512	0.162
	Sigmoid		0.031	0.042	0.629	0.782
	Softmax		0.026	0.031	1.422	0.487
	Tanh		0.035	0.044	0.638	0.782
MP-SPDZ (MPC)	GeLU	2	9.617	–	–	1578.9
		4	11.741	–	–	4736.69
		8	16.102	–	–	11052.3
		16	26.721	–	–	23683.4
		32	53.725	–	–	48945.8
MD-ML (MPC)	ReLU	2	0.202	0.378	3.433	19.985
MD-SONIC (MPC)	ReLU	2	6.361	6.695	7.195	0.353
		4	9.483	9.854	10.567	2.004
		8	17.024	17.288	17.928	4.676
		16	27.673	28.871	31.733	10.021
		32	61.917	65.131	>900	20.709
SMASH (MPC)	ReLU / GeLU / SiLU / Sigmoid	2	0.466	0.494	0.795	1.525
		4	0.846	0.827	1.121	4.577
		8	2.484	2.531	2.843	10.681
		16	10.122	10.127	10.409	22.888
		32	40.313	40.565	40.702	47.303

Table 3: Accuracy comparison on nonlinear functions.

Function	MP-SPDZ		SMASH	
	MSE	MAE	MSE	MAE
ReLU	3.34×10^{-10}	1.21×10^{-5}	1.96×10^{-11}	3.84×10^{-6}
Sigmoid	1.86×10^{-6}	4.88×10^{-4}	1.65×10^{-11}	3.31×10^{-6}
Tanh	1.53×10^{-5}	9.78×10^{-4}	9.05×10^{-12}	1.77×10^{-6}
GeLU	1.46×10^{-5}	8.24×10^{-4}	2.53×10^{-11}	4.19×10^{-6}
Sqrt	5.63×10^{-9}	6.83×10^{-5}	2.14×10^{-11}	3.92×10^{-6}
SiLU	5.38×10^{-5}	2.84×10^{-3}	4.01×10^{-11}	5.17×10^{-6}
Softmax	1.53×10^{-7}	1.23×10^{-4}	5.05×10^{-8}	1.08×10^{-4}

tional security. Fixed-point values use a 24-bit representation with 16 fractional bits. ElGamal-based threshold homomorphic encryption (ElGamal-THE) is instantiated over BLS12-381 via the `MCL` library [41].

Hardware and network. MPC experiments ran on servers with Intel Xeon Silver 4314 CPUs (64 cores) and 503 GiB RAM. Accuracy evaluation for `Qwen3-0.6B` used dual Intel Xeon Gold 6348 CPUs, 1.0 TiB RAM, and an NVIDIA A800 (80 GB) GPU. We evaluate three network configurations: **Local** (10 Gbps, 0.1 ms RTT), **LAN** (1 Gbps, 0.1 ms RTT), and **WAN** (200 Mbps, 100 ms RTT).

7.2 Nonlinear Function Evaluation

2PC vs. MPC. We evaluate maliciously secure protocols (Table 2). SHARK attains the lowest online latency and communication in 2PC, but relies on a *trusted dealer*; SMASH and the other baselines are dealer-free. MD-ML matches 2PC runtime locally but with higher communication, while MD-SONIC supports general MPC at substantial runtime cost. SMASH scales to 32 parties without trusted setup.

WAN sensitivity. Measured as the runtime increase from LAN to WAN, SMASH's constant two-round design adds only roughly 0.2-0.3 s; MD-ML slows by over 3 s and MD-SONIC degrades sharply beyond 16 parties. Hence SMASH is notably robust for geographically distributed deployments.

Numerical accuracy. Table 3 reports MSE and MAE: SMASH yields lower error than MP-SPDZ because LUT-based evaluation avoids iterative polynomial approximations and the truncation-induced error accumulation typical of arithmetic-based implementations.

Usage. SM-LUT supports arbitrary nonlinear functions via user-supplied fixed-point lookup tables (precision and step). It interoperates with arithmetic and Boolean shares and integrates into existing MPC pipelines through our maliciously secure share conversions.

Table 4: Comparison of conversions with MP-SPDZ.

Framework	Protocol	Party	Offline		Online	
			Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
MP-SPDZ	A2B	2	1.3141	50.400	0.0011	0.0004
		4	5.2141	349.20	0.0015	0.0014
		8	20.945	1697.9	0.0027	0.0032
		16	171.14	7910.4	0.0045	0.0067
		32	1513.7	34065	0.0683	0.0136
SMASH	A2L	2	0.0005	0.0005	0.0006	0.0003
		4	0.0015	0.0013	0.0010	0.0008
		8	0.0045	0.0031	0.0027	0.0018
		16	0.0138	0.0065	0.0078	0.0038
		32	0.0527	0.0133	0.0275	0.0078
MP-SPDZ	B2A	2	0.4252	22.746	0.0004	0.0003
		4	0.8483	68.243	0.0205	0.0010
		8	1.9197	159.25	0.0220	0.0024
		16	5.0332	341.26	0.0064	0.0050
		32	16.369	705.45	0.0971	0.0103
SMASH	L2A	2	0.0005	0.0005	0.0003	0.0002
		4	0.0014	0.0013	0.0004	0.0005
		8	0.0043	0.0031	0.0012	0.0011
		16	0.0136	0.0065	0.0046	0.0024
		32	0.0527	0.0133	0.0179	0.0050

7.3 Share Conversion Evaluation

We benchmark against MP-SPDZ because it implements *edaBits* [24] for efficient A2B/B2A. Since SMASH relies on A2L/L2A to bypass costly bit-decomposition (unlike the A2B/B2A path used by baselines), Table 4 shows that our approach drastically reduces overhead, improving offline costs by over $10^3\times$ while maintaining lower online latency. Furthermore, Table 5 compares our semi-honest secure A2B/B2A against Garg et al. [27]. Our protocols outperform the baseline, reducing B2A offline runtime by two orders of magnitude and consistently lowering communication costs.

7.4 Secure Inference Evaluation

Maliciously Secure Frameworks. For the Tiny/Base/Large model, Table 6 shows that SMASH achieves 15.8×, 18.5× and 18.9× speedup, and reduces communication by 976.9×, 1019.9× and 1038.8×, respectively. These gains indicate that SMASH effectively eliminates the dominant nonlinear bottleneck in maliciously secure inference, with cost scaling

Table 5: Comparison of conversions with Garg et al.'s [27].

Framework	Protocol	Party	Offline		Online	
			Time (ms)	Comm. (MB)	Time (ms)	Comm. (MB)
Garg et al.'s	A2B	2	0.3704	16.793	0.0004	0.0862
		4	0.5159	40.012	0.0005	0.1293
		8	1.1680	86.815	0.0007	0.1507
		16	3.3524	164.59	0.0014	0.1585
		32	9.7554	263.86	0.0022	0.1617
SMASH	A2B	2	0.4859	9.6333	0.0004	0.0833
		4	0.6920	26.613	0.0004	0.1249
		8	1.8941	64.132	0.0006	0.1457
		16	4.1297	107.04	0.0010	0.1561
		32	11.081	189.82	0.0021	0.1613
Garg et al.'s	B2A	2	0.1773	5.6096	0.0006	0.0313
		4	0.2167	11.033	0.0014	0.0469
		8	0.3712	20.028	0.0330	0.0547
		16	1.2160	32.780	0.0165	0.0586
		32	4.1520	53.288	0.0450	0.0605
SMASH	B2A	2	0.0096	1.3520	0.0008	0.0039
		4	0.0083	3.9018	0.0009	0.0059
		8	0.0171	9.9567	0.0013	0.0068
		16	0.0323	10.668	0.0020	0.0073
		32	0.0608	11.024	0.0040	0.0076

Table 6: Comparison of maliciously secure frameworks for end-to-end inference.

Framework	Model	Total Nonlinear Cost		Ops.	Time (s)	Comm. (GB)	Count
		Time (s)	Comm. (GB)				
MP-SPDZ (Local)	Tiny	750.3	104.9	GELU	72.5	10.105	4
				Softmax	56.6	7.651	4
				LayerNorm	19.4	2.827	12
	Base	12106.0	1613.5	GELU	459.2	60.629	12
				Softmax	176.1	22.954	12
				LayerNorm	124.4	16.959	36
	Large	30229.7	4057.8	GELU	614.0	80.839	24
				Softmax	233.2	30.605	16
				LayerNorm	163.3	22.612	72
SMASH (WAN)	Tiny	47.3	0.1	GELU	4.1	0.010	4
				Softmax	4.1	0.010	4
				LayerNorm	1.1	0.002	12
	Base	651.9	1.5	GELU	24.0	0.059	12
				Softmax	12.2	0.029	12
				LayerNorm	6.0	0.015	36
	Large	1599.1	3.9	GELU	31.6	0.078	24
				Softmax	16.2	0.039	16
				LayerNorm	8.0	0.020	72

linearly in the number of nonlinear invocations and remaining independent of function complexity.

Semi-honest Secure Frameworks. For frameworks where end-to-end nonlinear costs are not directly reported, the results are derived following the methodology described in the original works. In particular, SHAFT's nonlinear cost is estimated by aggregating its micro-benchmark results using the authors' stated cost model (computation $+2\times$ communication$/$bandwidth $+$rounds$\times$ latency), while BumbleBee's nonlinear overhead is obtained by profiling and summing all nonlinear operations. Table 7, SMASH significantly outperforms existing semi-honest systems. For the Base model, SMASH reduces communication and runtime by $76\times$ and $67\times$ versus BOLT, and achieves over $13\times$ lower communication than both BumbleBee and SHAFT.

Malicious vs. Semi-honest Overhead. Comparing malicious to semi-honest settings, SMASH's nonlinear time for Tiny/Base/Large models increases $7\times/15\times/16\times$, with $4\times$ more communication. This overhead, mainly from lightweight ZKPoKs, scales linearly with nonlinear invocations and remains independent of function complexity, representing an acceptable cost for malicious security.

Table 7: Comparison of semi-honest secure frameworks for end-to-end inference in WAN.

Framework	Model	Total Nonlinear Cost		Ops.	Time (s)	Comm. (GB)
		Time (s)	Comm. (GB)			
BOLT	Base	2743.980	24.170	GeLU	766.810	8.623
				Softmax	869.460	8.482
				LayerNorm	1107.710	6.991
BumbleBee	Base	1543.001	4.216	–	–	–
SHAFT	Base	>400	>4.160	GeLU	51.132	4.148
				Softmax	365.283	0.003
SMASH	Tiny	5.985	0.021	GeLU	1.519	0.008
				Softmax	1.519	0.008
				LayerNorm	2.947	0.006
	Base	40.961	0.316	GeLU	15.297	0.141
				Softmax	8.831	0.070
				LayerNorm	16.833	0.105
	Large	93.937	0.781	GeLU	39.128	0.375
				Softmax	14.659	0.125
				LayerNorm	40.150	0.281

Table 8: Performance under different security models (taking 64 tokens for example).

Function	Malicious Comm. (MB)		Semi-honest	
	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
Softmax	4.180	10.000	0.381	2.000
Sin	0.703	1.250	0.223	0.250
SiLU	12.240	30.000	0.737	6.000
RMSNorm	0.702	0.244	0.607	0.048
Cos	0.704	1.250	0.223	0.250

Table 9: Performance of Qwen3-0.6B on GSM8K test set under two inference modes.

Thinking				NoThinking			
Plaintext		SM-LUT-based		Plaintext		SM-LUT-based	
Accuracy	ATL	Accuracy	ATL	Accuracy	ATL	Accuracy	ATL
59.59%	1954	55.16%	2354	57.24%	227	54.85%	230

All inferences are performed with a maximum generation length of 8192 tokens. "ATL" denotes the average token length generated per sample.

7.5 Applicability to LLMs

We measure the performance of SM-LUT on the nonlinear operations of Qwen3-0.6B [58]. We evaluated for different tokens, and the performance trends are consistent across all settings. Table 8 reports representative results for 64 tokens due to space constraints. Across all operations, the malicious setting incurs a $0.2\times$-$15.6\times$ increase in runtime and $4\times$ increase in communication compared to the semi-honest variant. Table 9 evaluates end-to-end secure inference on Qwen3-0.6B over the GSM8K benchmark. SM-LUT-based inference achieves accuracy close to plaintext execution, with

a relative drop of 4.43% / 2.39% in Thinking / NoThinking mode. This degradation is expected, as small approximation errors introduced by SM-LUT accumulate over long-context generation. Nevertheless, the model preserves reasoning capability and output quality, demonstrating that SM-LUT strikes a favorable balance between efficiency and model fidelity.

8 Conclusion and Future Work

In this paper, we presented SMASH, a scalable hybrid MPC framework designed to bridge the gap between theoretical malicious security and the practical demands of LLM inference. By reformulating nonlinear evaluations into structured rotations in the DFT domain, SMASH successfully reduces the communication burden from a quadratic to a linear dependency on the number of parties. Our evaluation confirms that SMASH effectively eliminates the nonlinear bottleneck that has long plagued maliciously secure systems, maintaining high inference fidelity even for modern models.

Broader Impact and Future Work. The architectural principles of SMASH extend beyond inference. The linear scalability of SM-LUT suggests a promising path toward *maliciously secure collaborative training*, where gradients and activations could be evaluated with similar efficiency. Moreover, SM-LUT could be extended to accelerate maliciously secure A2B/B2A (e.g., edaBit [24]) by replacing costly Boolean verification circuits with efficient LUT-based checks. Future optimizations could explore *value interpolation* across multiple sub-LUTs to further enhance throughput at a marginal cost to precision. Additionally, integrating SMASH with *hardware-accelerated primitives* (e.g., GPU-friendly DFT implementations) could push the boundaries of real-time private intelligence even further. Ultimately, SMASH demonstrates that robust protection against malicious adversaries is no longer an unaffordable luxury, but a viable standard for next-generation privacy-preserving AI.

Acknowledgments

We thank the reviewers and editors for their constructive comments. Yunlv Lv, Rui Zhang, and Ziyi Wan were supported by the National Natural Science Foundation of China (Nos. 62172411, U2336205) and the Beijing Municipal Science and Technology Project (No. Z231100005923047). Jiangtao Li was supported by NSFC No. 62402305.

Ethics Considerations

We present a scalable, maliciously secure MPC framework for cross-organization ML and LLM inference (e.g., healthcare, finance, cross-border compliance). We consider relevant stakeholders including data subjects, deploying institutions, regulators, auditors, researchers, and society, and weigh potential benefits, such as privacy-preserving collaboration and reduced centralization, against potential harms, including malicious misuse and inadvertent privacy leakage. To mitigate risks, all experiments used only synthetic or publicly available datasets; the protocol offers abort security and publicly verifiable zero-knowledge proofs of knowledge; and deployments can adopt optional accountability mechanisms (e.g., verifiable key escrow). After assessing residual risks versus societal benefits (safer collaborative computation, reproducibility, wider access to privacy-preserving techniques), we concluded that releasing the research is justified.

Open Science

We release the artifact at `https://zenodo.org/records/18494350` as `SMASH-main.zip` and `Baselines.zip`. `SMASH-main.zip` includes the C++ codebase, build files, `Test_scripts/` and `Results/`; `Baselines.zip` contains baseline implementations. A top-level README lists build commands, environment, dataset sources, and exact commands to reproduce the main figures and tables.

References

[1] X. Bai, H. Wang, L. Ma, Y. Xu, J. Gan, Z. Fan, F. Yang, K. Ma, J. Yang, S. Bai, et al. Advancing COVID-19 diagnosis with privacy-preserving collaboration in artificial intelligence. Nat. Mach. Intell., 3(12):1081–1089, 2021.

[2] D. Beaver, S. Micali, and P. Rogaway. The round complexity of secure protocols. In STOC, pages 503–513, 1990.

[3] G. Beck, A. Goel, A. Hegde, A. Jain, Z. Jin, and G. Kaptchuk. Scalable multiparty garbling. In CCS, pages 2158–2172, 2023.

[4] M. Bellare and G. Neven. Multi-signatures in the plain public-key model and a general forking lemma. In CCS, pages 390–399, 2006.

[5] R. Bendlin, I. Damgård, C. Orlandi, and S. Zakarias. Semi-homomorphic encryption and multiparty computation. In EUROCRYPT, pages 169–188, 2011.

[6] L. Braun, D. Demmler, T. Schneider, and O. Tkachenko. MOTION–a framework for mixed-protocol multi-party computation. ACM Transactions on Privacy and Security, 25(2):1–35, 2022.

[7] A. Brüggemann, R. Hundt, T. Schneider, A. Suresh, and H. Yalame. FLUTE: fast and secure lookup table evaluations. In S&P, pages 515–533, 2023.

[8] R. Canetti. Security and composition of multiparty cryptographic protocols. J. Cryptol., 13:143–202, 2000.

[9] R. Canetti, R. Gennaro, S. Goldfeder, N. Makriyannis, and U. Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In CCS, pages 1769–1787, 2020.

[10] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, et al. Extracting training data from large language models. In USENIX, pages 2633–2650, 2021.

[11] E. Chen, J. Zhu, A. Ozdemir, R. S. Wahby, F. Brown, and W. Zheng. Silph: A framework for scalable and accurate generation of hybrid MPC protocols. In S&P, pages 848–863, 2023.

[12] Y. Chen, X. Meng, Z. Shi, Z. Ning, and J. Lin. SecureTLM: Private inference for transformer-based large model with MPC. Information Sciences, 667:120429, 2024.

[13] S. S. Chow, C. Ma, and J. Weng. Zero-knowledge argument for simultaneous discrete logarithms. Algorithmica, 64:246–266, 2012.

[14] CISA. AI data security: Best practices for securing data used to train & operate AI systems, 2024. https://www.cisa.gov/resources-tools/resources/ai-data-security-best-practices-securing-data-used-train-operate-ai-systems. Accessed: 2025-06-04.

[15] Communications of the ACM. Mining financial data without actually seeing it can detect fraud. https://cacm.acm.org/news/mining-financial-data-without-actually-seeing-it-can-detect-fraud/, 2024. [Online; accessed 2025-08-09].

[16] I. Damgård, D. Escudero, T. Frederiksen, M. Keller, P. Scholl, and N. Volgushev. New primitives for actively-secure MPC over rings with applications to private machine learning. In S&P, pages 1102–1120, 2019.

[17] I. Damgård, J. B. Nielsen, M. Nielsen, and S. Ranellucci. The tinytable protocol for 2-party secure computation, or: Gate-scrambling revisited. In CRYPTO, pages 167–187, 2017.

[18] I. Damgård, V. Pastro, N. Smart, and S. Zakarias. Multiparty computation from somewhat homomorphic encryption. In CRYPTO, pages 643–662, 2012.

[19] I. Damgård and R. Zakarias. Fast oblivious AES a dedicated application of the MiniMac protocol. In International Conference on Cryptology in Africa, pages 245–264, 2016.

[20] S. Das and L. Ren. Adaptively secure BLS threshold signatures from DDH and CO-CDH. In CRYPTO, pages 251–284, 2024.

[21] S. de Hoogh, B. Schoenmakers, B. Škorić, and J. Villegas. Verifiable rotation of homomorphic encryptions. In PKC, pages 393–410, 2009.

[22] D. Demmler, T. Schneider, and M. Zohner. ABY-a framework for efficient mixed-protocol secure two-party computation. In NDSS, 2015.

[23] T. ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. TIT, 31(4):469–472, 1985.

[24] D. Escudero, S. Ghosh, M. Keller, R. Rachuri, and P. Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In CRYPTO, pages 823–852, 2020.

[25] A. Fiat and A. Shamir. How to prove yourself: practical solutions to identification and signature problems. In CRYPTO, pages 186–194, 1986.

[26] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, 2019.

[27] R. Garg, K. Yang, J. Katz, and X. Wang. Scalable mixed-mode MPC. In S&P, pages 523–541, 2024.

[28] R. Geva, A. Gusev, Y. Polyakov, L. Liram, O. Rosolio, A. Alexandru, N. Genise, M. Blatt, Z. Duchin, B. Waissengrin, et al. Collaborative privacy-preserving analysis of oncological data using multiparty homomorphic encryption. IACR Cryptol. ePrint Arch., 120(33):e2304415120, 2023.

[29] Google Cloud. To help combat fraud, google cloud and swift pioneer advanced ai and federated learning tech. https://cloud.google.com/blog/products/identity-security/google-cloud-and-swift-pioneer-advanced-ai-and-federated-learning-tech, 2024. [Online; accessed 2025-08-09].

[30] J. Groth. On the size of pairing-based non-interactive arguments. In EUROCRYPT, pages 305–326, 2016.

[31] K. Gupta, N. Chandran, D. Gupta, J. Katz, and R. Sharma. Shark: Actively secure inference using function secret sharing. In S&P, pages 2472–2490, 2025.

[32] D. Heath, V. Kolesnikov, and L. K. Ng. Garbled circuit lookup tables with logarithmic number of ciphertexts. In EUROCRYPT, pages 185–215, 2024.

[33] Y. Ishai, E. Kushilevitz, S. Meldgaard, C. Orlandi, and A. Paskin-Cherniavsky. On the power of correlated randomness in secure computation. In TCC, pages 600–620, 2013.

[34] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In USENIX, pages 1651–1669, 2018.

[35] A. Y. Kei and S. S. Chow. Shaft: Secure, handy, accurate, and fast transformer inference. In NDSS, 2025.

[36] M. Keller. MP-SPDZ: A versatile framework for multi-party computation. In CCS, pages 1575–1590, 2020.

[37] M. Keller, E. Orsini, and P. Scholl. MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In CCS, pages 830–842, 2016.

[38] D. Li, R. Shao, H. Wang, H. Guo, E. P. Xing, and H. Zhang. MPCFormer: fast, performant and private transformer inference with MPC. arXiv preprint arXiv:2211.01452, 2022.

[39] J. Liu, M. Juuti, Y. Lu, and N. Asokan. Oblivious neural network predictions via minionn transformations. In CCS, pages 619–631, 2017.

[40] W. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, C. Hong, K. Ren, T. Wei, and W. Chen. BumbleBee: Secure two-party inference framework for large transformers. In NDSS, 2025.

[41] S. Mitsunari. MCL: A portable and fast pairing-based cryptography library. Accessed: Jun, 20, 2023.

[42] P. Mohassel and P. Rindal. ABY3: A mixed protocol framework for machine learning. In CCS, pages 35–52, 2018.

[43] P. Mohassel and Y. Zhang. SecureML: A system for scalable privacy-preserving machine learning. In S&P, pages 19–38, 2017.

[44] C. Mouchet, J. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux. Multiparty homomorphic encryption from ring-learning-with-errors. Proc. Privacy Enhancing Tech. Symp., 2021(4):291–311, 2021.

[45] L. K. Ng and S. S. Chow. GForce:GPU-Friendly oblivious and rapid neural network inference. In USENIX, pages 2147–2164, 2021.

[46] V. Nikolaenko, U. Weinsberg, S. Ioannidis, M. Joye, D. Boneh, and N. Taft. Privacy-preserving ridge regression on hundreds of millions of records. In S&P, pages 334–348, 2013.

[47] E. Orsini, N. P. Smart, and F. Vercauteren. Overdrive2k: Efficient secure mpc over from somewhat homomorphic encryption. In CT-RSA, pages 254–283. Springer, 2020.

[48] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In S&P, pages 4753–4771, 2024.

[49] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In CRYPTO, pages 129–140, 1991.

[50] J.-J. Quisquater et al. How to explain zero-knowledge protocols to your children. In EUROCRYPT, pages 628–631, 1989.

[51] D. Rathee, M. Rathee, N. Kumar, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma. Cryptflow2: Practical 2-party secure inference. In CCS, pages 325–342, 2020.

[52] M. S. Riazi, C. Weinert, O. Tkachenko, E. M. Songhori, T. Schneider, and F. Koushanfar. Chameleon: A hybrid secure computation framework for machine learning applications. In CCS, pages 707–721, 2018.

[53] D. Rotaru and T. Wood. Marbled circuits: Mixing arithmetic and boolean circuits with active security. In INDOCRYPT, pages 227–249, 2019.

[54] M. B. Santos, D. Mouris, M. Ugurbil, S. Jarecki, J. Reis, S. Sengupta, and M. de Vega. Curl: Private LLMs through wavelet-encoded look-up tables. Cryptology ePrint Archive, 2024.

[55] C.-P. Schnorr. Efficient signature generation by smart cards. J. Cryptol., 4:161–174, 1991.

[56] S. Wagh, S. Tople, F. Benhamouda, E. Kushilevitz, P. Mittal, and T. Rabin. Falcon: Honest-majority maliciously secure framework for private deep learning. arXiv preprint arXiv:2004.02229, 2020.

[57] G. M. Weber, C. Hong, Z. Xia, N. P. Palmer, P. Avillach, S. Lyi, M. S. Keller, S. N. Murphy, A. Gutierrez-Sacristan, C.-L. Bonzel, et al. International comparisons of laboratory values from the 4ce collaborative to predict COVID-19 mortality. NPJ digital medicine, 5(1):74, 2022.

[58] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

[59] B. Yuan, S. Yang, Y. Zhang, N. Ding, D. Gu, and S.-F. Sun. MD-ML: Super fast privacy-preserving machine learning for malicious security with a dishonest majority. In USENIX, pages 2227–2244, 2024.

[60] M. Zhang, Y. Chen, C. Yao, and Z. Wang. Sigma protocols from verifiable secret sharing and their applications. In ASIACRYPT, pages 208–242, 2023.

[61] Y. Zhang, X. Chen, Y. Dong, Q. Zhang, R. Hou, Q. Liu, and X. Chen. Md-sonic: Maliciously-secure outsourcing neural network inference with reduced online communication. TIFS, 2025.

A Extended Preliminaries

Functionality $\mathcal{F}_{\text{Con}}$

This functionality extends $\mathcal{F}_{\text{ABB}}$ with the following additional commands:

Boolean-to-Arithmetic (B2A): Upon receiving (B2A, $\mathbf{id}_1$, id_2) from all parties, where $\mathbf{id}_1$ is a vector of l fresh identifiers, retrieve ($\mathbf{id}_1$, bool, x), store (id_2, arith, x).

Arithmetic-to-Boolean (A2B): Upon receiving (A2B, id_1, $\mathbf{id}_2$) from all parties, retrieve (id_1, arith, x) and store ($\mathbf{id}_2$, bool, x_i), where $\mathbf{id}_2$ is a vector of l fresh identifiers and $x = \sum_{i=0}^{l-1} x_i 2^i \bmod q$.

LUT-to-Encryption (L2E): Upon receiving (L2E, id_1, id_2, id_3) from all parties, where (id_1, lut) and (id_2, enc) are present in memory, retrieve (id_1, lut, x) and store (id_3, dec, x).

Encryption-to-LUT (E2L): Upon receiving (E2L, id_1, id_2, id_3) from all parties, where (id_1, dec) is present in memory, retrieve (id_1, dec, x) and store (id_2, lut, x) and (id_3, enc, c), where c is a fresh HE encryption of x.

Arithmetic-to-LUT (A2L): Upon receiving (A2L, id_1, id_2, id_3) from all parties, retrieve (id_1, arith, x) and store (id_2, lut, x) and (id_3, enc, c), where c is a fresh HE encryption of x.

LUT-to-Arithmetic (L2A): Upon receiving (L2A, id_1, id_2) from all parties, retrieve (id_1, lut, x) and store (id_2, arith, x).

Figure 13: Functionality for conversion black box

B Additonal ZKPoKs

We provide here the full descriptions of the additional ZKPoKs Π_{Enc}, Π_{Ran}, and Π_{Cip}, which are only summarized in the main text. All three follow the Schnorr-style three-move Σ-protocol template. Therefore, their completeness, special soundness, and honest-verifier zero-knowledge properties can be established by standard arguments and are omitted. By the standard analysis of Schnorr-like Σ-protocols [55], each of these protocols is complete, possesses special soundness, and is honest-verifier zero-knowledge.

Π_{Enc}. Protocol for relation $\mathsf{R}_{\text{Enc}} = \{(g,h,y_1,y_2;x,r) \mid y_1 = g^r, y_2 = g^x h^r\}$: The prover samples $r_1, r_2 \leftarrow \mathbb{Z}_q$ and sends $t_1 = g^{r_1}$, $t_2 = g^{r_2} h^{r_1}$. The verifier sends a challenge $\lambda \leftarrow \mathbb{Z}_q$. The prover responds with $s_x = r_2 + \lambda x$, $s_r = r_1 + \lambda r$. The verifier accepts if $g^{s_r} = t_1 y_1^{\lambda}$ and $g^{s_x} h^{s_r} = t_2 y_2^{\lambda}$.

Π_{Ran}. Protocol for relation $\mathsf{R}_{\text{Ran}} = \{(p,g,g_1,h,y_1,y_2,y_3; x,r) \mid y_1 = g^r, y_2 = g^x g_1^r, y_3 = g^x h^r, x \in [0,p)\}$: The prover samples $r_1 \leftarrow \mathbb{Z}_q$, $r_2 \leftarrow \mathbb{Z}_p$ and sends $t_1 = g^{r_1}$, $t_2 = g^{r_2} g_1^{r_1}$, $t_3 = g^{r_2} h^{r_1}$. The verifier sends a challenge $\lambda \leftarrow \mathbb{Z}_p$. The prover responds with $s_x = r_2 + \lambda x$, $s_r = r_1 + \lambda r$. The verifier accepts if $g^{s_r} = t_1 y_1^{\lambda}$, $s_x < p$, $g^{s_x} g_1^{s_r} = t_2 y_2^{\lambda}$, and $g^{s_x} h^{s_r} = t_3 y_3^{\lambda}$.

Π_{Cip}. Protocol for relation $\mathsf{R}_{\text{Cip}} = \{(g,h,g_1,y_1,y_2,y_3;x,r) \mid y_1 = g^r, y_2 = g^x h^r, y_3 = g_1^x h^r\}$: The prover samples $r_1, r_2 \leftarrow \mathbb{Z}_q$ and sends $t_1 = g^{r_1}$, $t_2 = g^{r_2} h^{r_1}$, $t_3 = g_1^{r_2} h^{r_1}$. The verifier sends a challenge $\lambda \leftarrow \mathbb{Z}_q$. The prover responds with $s_x = r_2 + \lambda x$, $s_r = r_1 + \lambda r$. The verifier accepts if $g^{s_r} = t_1 y_1^{\lambda}$, $g^{s_x} h^{s_r} = t_2 y_2^{\lambda}$, and $g_1^{s_x} h^{s_r} = t_3 y_3^{\lambda}$.

C Formal Proofs of Our Protocols

For all proofs, we denote $\mathbf{H}$ be the set of honest parties and $\mathbf{A}$ be the component thereof.

C.1 Proof of Theorem 1

Proof. According to the syntax defined in Section 2, we prove that our protocol satisfies *completeness*, *special soundness*, and *honest verifier zero-knowledge* by leveraging the multiple-forking lemma (Lemma 1) formalized by Chow et al. [13].

Lemma 1. *Fix $\alpha \in \mathbb{Z}^+$ and a set S such that $|S| \geq 2$. Let $\mathcal{Y}$ be a randomized algorithm that, given an input x and a sequence of elements $s_1, \ldots, s_\alpha \in S$, outputs a triple (I, J, σ), where $0 \leq J \leq I \leq \alpha$, and a string σ. Let G be a randomized algorithm that takes no input and returns a string. Define the forking strategy $F_{\mathcal{Y},n}$ associated with $\mathcal{Y}$ and an odd integer $n \geq 1$ as in algorithm $F_{\mathcal{Y},n}$. Then we have:*

$$acc = \Pr[x \leftarrow_R G; s_1, \ldots, s_\alpha \leftarrow_R S; (I, J, \sigma^{(0)}) \leftarrow_R \mathcal{Y}(x, s_1, \ldots, s_\alpha; \rho) : I \geq 1 \wedge J \geq 1];$$

$$frk = \Pr[x \leftarrow_R G; F_{\mathcal{Y},n}(x) \neq \perp];$$

$$frk \geq acc\left(\frac{acc^n}{\alpha^{2n}} - \frac{n}{|S|}\right), \text{i.e., } acc \leq \sqrt[n+1]{\alpha^{2n} frk} + \sqrt[n+1]{\frac{n\alpha^{2n}}{|S|}}.$$

The forking strategy $F_{\mathcal{Y},n}$, as described in algorithm $F_{\mathcal{Y},n}$, plays a central role in the proof of Theorem 1. Each forking corresponds to modifying the input challenge sequence s, and the number of such forkings is parameterized by n.

The algorithm $F_{\mathcal{Y},n}$. On input x, proceed as follows.

1) Sample random ρ for $\mathcal{Y}$ and draw $s_1, \ldots, s_\alpha \leftarrow_R S$. Let $(I, J, \sigma(0)) \leftarrow \mathcal{Y}(x, s_1, \ldots, s_\alpha; \rho)$. If $I = 0$ or $J = 0$, output $\perp$.

2) Initialize $i \leftarrow 1$ and fork on index I: resample $s_I^{(i)}, \ldots, s_\alpha^{(i)} \leftarrow_R S$ and compute $(I^{(i)}, J^{(i)}, \sigma^{(i)})$ using the same ρ. Abort if $(I^{(i)}, J^{(i)}) \neq (I, J)$ or $s_I^{(i)} = s_I$. Set $i \leftarrow 2$.

3) While $i < n$, perform two alternating forks. Fork on index J: resample $s_J^{(i)}, \ldots, s_\alpha^{(i)}$, check index consistency and collision with the previous fork; abort on failure and set $i \leftarrow i+1$. Fork on index I: resample $s_I^{(i)}, \ldots, s_\alpha^{(i)}$, perform the same checks, and increment i.

4) Output $\sigma(0), \ldots, \sigma(n)$.

Completeness. Define the communication transcript between a prover and a verifier as $(\widetilde{C}, \{c_{k+1}, M_k\}_{k=0}^{\bar{n}-1}, C, \lambda, \sigma, \eta, \psi, \{\mu_k, \nu_k, \rho_k\}_{k=0}^{\bar{n}-1})$. The equations $\{d_k = a_k^{\beta^k} g^{s_k}, e_k = b_k^{\beta^k} h^{s_k}\}_{k=0}^{\bar{n}-1}$ implies that the verification equation always holds: $\widetilde{h}^{\eta} = \widetilde{C}(c_{\bar{n}}/g)^{\lambda}$, $(\prod_{k=0}^{\bar{n}-1} c_k^{y_k})^{\sigma} \widetilde{h}^{\psi} = C(\prod_{k=0}^{\bar{n}-1} c_{k+1}^{y_k})^{\lambda}$ and $g^{z_1\mu_k + z_2\nu_k} \widetilde{h}^{z_1\rho_k} a_k^{z_2\mu_k} b_k^{z_3\mu_k} h^{z_3\nu_k} = M_k c_k^{z_1\lambda} d_k^{z_2\lambda} e_k^{z_3\lambda}$ for all $k \in [0, \bar{n}-1]$.

Special Soundness. Inspired by the proof strategy proposed by Chow et al. [13], we establish the special soundness of our protocol within the RO model, where the hash function H is modeled as a RO. Suppose there exists an adversary $\mathcal{A}$ capable of breaking the special soundness of the protocol with non-negligible probability ε. That is, $\mathcal{A}$ can produce values such that the verifier V accepts $d_k = a_k^{m_1} g^{n_1}$ and $e_k = b_k^{m_2} h^{n_2}$ even though $(m_1, n_1) \neq (m_2, n_2)$ or $m_1 = m_2 \neq p^k$ for any $p \in \mathbb{Z}_q$. Then, one can construct an algorithm $\mathcal{B}$ that solves the discrete logarithm (DL) problem with advantage at least $\varepsilon^* \geq \mathtt{frk} \geq \varepsilon\left(\frac{\varepsilon^5}{(Q_H Q_C)^{10}} - \frac{5}{|\mathbb{Z}_q^*|}\right)$, where Q_H denotes the number of hash queries made by $\mathcal{A}$ and Q_C the number of interactions between $\mathcal{A}$ and the verifier.

Assuming $\mathcal{A}$ can succeed with probability ε within time t, we design $\mathcal{B}$ to solve the DL problem for a challenge (g, h) by computing $\log_g h$ with probability at least ε^* within time t^*. Algorithm $\mathcal{B}$ embeds the DL challenge into the public parameters and emulates the RO $H(\cdot,\cdot)$ using a list L_H (each new query is answered with a fresh random element from $\mathbb{Z}_q^*$, while repeated queries yield consistent responses). $\mathcal{B}$ provides $\mathcal{A}$ with public parameters $param = (g, h, \tilde{h}, H)$, where $\tilde{h}$ is sampled uniformly at random from $\mathbb{Z}_q$. During execution, $\mathcal{A}$ outputs $a_k, b_k, d_k, e_k{}_{k=0}^{\bar{n}-1}$, attempting to cause the verifier to accept a malformed proof. Suppose for some index k, the verifier accepts a pair (d_k, e_k) with the aforementioned malformed structure. At this point, $\mathcal{B}$ selects a challenge $\lambda \leftarrow_R \mathbb{Z}q^*$ and invokes the generalized forking algorithm $F_{\mathcal{Y},5}(param)$, aligned with the multiple-forking lemma. The wrapper algorithm $\mathcal{Y}$ simulates the interaction of $\mathcal{A}$ with the oracle and verifier, tracking the relevant queries and challenges. By running the multiple-forking algorithm, either $\mathcal{B}$ aborts prematurely (i.e., $F_{\mathcal{Y},5}(param)$ returns $\perp$), or gets six "cheating" transcripts (the output of the interaction between P and V in the real protocol which corresponds to the interaction between $\mathcal{A}$ and $\mathcal{B}$ in the proof) in the form of

$$(\widetilde{C}^{(i)}, \{c_{k+1}^{(i)}, M_k^{(i)}\}_{k=0}^{\bar{n}-1}, C^{(i)}, \lambda^{(i)}, \sigma^{(i)}, \eta^{(i)}, \psi^{(i)}, \{\mu_k^{(i)}, \nu_k^{(i)}, \rho_k^{(i)}\}_{k=0}^{\bar{n}-1})$$
$$(\widetilde{C}^{(i)}, \{c_{k+1}^{(i)}, M_k^{(i)}\}_{k=0}^{\bar{n}-1}, C^{(i)}, \hat{\lambda}^{(i)}, \hat{\sigma}^{(i)}, \hat{\eta}^{(i)}, \hat{\psi}^{(i)}, \{\hat{\mu}_k^{(i)}, \hat{\nu}_k^{(i)}, \hat{\rho}_k^{(i)}\}_{k=0}^{\bar{n}-1})$$

for $i = 1, 2, 3$. From the response equations we have, for all k, $\mu_k^{(i)} = u_k + \lambda^{(i)}\beta^k, \hat{\mu}_k^{(i)} = u_k + \hat{\lambda}^{(i)}\beta^k, \nu_k^{(i)} = v_k + \lambda^{(i)} s_k, \hat{\nu}_k^{(i)} = v_k + \hat{\lambda}^{(i)} s_k$. Hence, an efficient extractor exists that outputs the witness $\beta^k = (\mu_k^{(i)} - \hat{\mu}_k^{(i)})/(\lambda^{(i)} - \hat{\lambda}^{(i)}), s_k = (\nu_k^{(i)} - \hat{\nu}_k^{(i)})/(\lambda^{(i)} - \hat{\lambda}^{(i)})$. As all these transcripts correspond to valid proofs, we have the following equation hold in particular,

$$\begin{aligned}
&\widetilde{C}^{(i)} = \widetilde{h}^{\eta^{(i)}} / \left((c_{\bar{n}}^{(i)}/g)^{\lambda^{(i)}}\right) = \widetilde{h}^{\hat{\eta}^{(i)}} \left((c_{\bar{n}}^{(i)}/g)^{\hat{\lambda}^{(i)}}\right) \\
&M_k^{(i)} = g^{z_1^{(i)}\hat{\mu}_k^{(i)} + z_2^{(i)}\hat{\nu}_k^{(i)}} \widetilde{h}^{z_1^{(i)}\hat{\rho}_k^{(i)}} a_k^{z_2^{(i)}\hat{\mu}_k^{(i)}} b_k^{z_3^{(i)}\hat{\mu}_k^{(i)}} h^{z_3^{(i)}\hat{\nu}_k^{(i)}} / \left(c_k^{z_1^{(i)}\hat{\lambda}^{(i)}} d_k^{z_2^{(i)}\hat{\lambda}^{(i)}} e_k^{z_3^{(i)}\hat{\lambda}^{(i)}}\right) \\
&C^{(i)} = \frac{\left(\prod_{k=0}^{\bar{n}-1} c_k^{(i)}\right)^{\sigma^{(i)}} \widetilde{h}^{\psi^{(i)}}}{\left(\prod_{k=0}^{\bar{n}-1} c_{k+1}^{(i)}\right)^{\lambda^{(i)}}} = \frac{\left(\prod_{k=0}^{\bar{n}-1} c_k^{(i)}\right)^{\hat{\sigma}^{(i)}} \widetilde{h}^{\hat{\psi}^{(i)}}}{\left(\prod_{k=0}^{\bar{n}-1} c_{k+1}^{(i)}\right)^{\hat{\lambda}^{(i)}}}
\end{aligned} \tag{1}$$

where $z^{(i)}$ is the RO reply for $H(i, \{a_j, b_j, d_j, e_j\}_{j=0}^{\bar{n}-1})$ for $i = 1, 2, 3$, and $y_k^{(i)}$ is the RO reply for $H(\{a_j, b_j, d_j, e_j\}_{j=0}^{\bar{n}-1}, k)$ for $i = 1, 2, 3$ and $k \in [0, \bar{n}-1]$ respectively.

Let $\Delta\lambda^{(i)} = \lambda^{(i)} - \hat{\lambda}^{(i)}$, $\Delta\sigma^{(i)} = \sigma^{(i)} - \hat{\sigma}^{(i)}$, $\Delta\eta^{(i)} = \eta^{(i)} - \hat{\eta}^{(i)}$, $\Delta\psi^{(i)} = \psi^{(i)} - \hat{\psi}^{(i)}$, $\Delta\mu_k^{(i)} = \mu_k^{(i)} - \hat{\mu}_k^{(i)}$, $\Delta\nu_k^{(i)} = \nu_k^{(i)} - \hat{\nu}_k^{(i)}, \Delta\rho_k^{(i)} = \rho_k^{(i)} - \hat{\rho}_k^{(i)}, \theta^{(i)} = \Delta\sigma^{(i)}/\Delta\lambda^{(i)}, \zeta^{(i)} = \Delta\eta^{(i)}/\Delta\lambda^{(i)}, \delta^{(i)} = \Delta\psi^{(i)}/\Delta\lambda^{(i)}$, and $\gamma_k^{(i)} = \Delta\mu_k^{(i)}/\Delta\lambda^{(i)}$, $\tau_k^{(i)} = \Delta\nu_k^{(i)}/\Delta\lambda^{(i)}, \xi_k^{(i)} = \Delta\rho_k^{(i)}/\Delta\lambda^{(i)}$, $\bar{t}_k^{(i)} = \xi_{k+1}^{(i)} - \xi_k^{(i)}\theta^{(i)}$ for $i = 1, 2, 3$. From the equations (1), we can obtain

$$\begin{aligned}
&\widetilde{h}^{\zeta^{(i)}} = \frac{c_{\bar{n}}^{(i)}}{g}, \left(\prod_{k=0}^{\bar{n}-1} c_k^{(i)}\right)^{\theta^{(i)}} \widetilde{h}^{\delta^{(i)}} = \prod_{k=0}^{\bar{n}-1} c_{k+1}^{(i)} \\
&g^{z_1^{(i)}\gamma_k + z_2^{(i)}\tau_k} \widetilde{h}^{z_1^{(i)}\xi_k} a_k^{z_2^{(i)}\gamma_k} b_k^{z_3^{(i)}\gamma_k} h^{z_3^{(i)}\tau_k} = (c_k^{(i)})^{z_1^{(i)}} d_k^{z_2^{(i)}} e_k^{z_3^{(i)}}
\end{aligned} \tag{2}$$

We define $\bar{u}_k^{(i)} = (\lambda^{(i)}\hat{\mu}_k^{(i)} - \hat{\lambda}^{(i)}\mu_k^{(i)})/\Delta\lambda^{(i)}$, $\bar{v}_k^{(i)} = (\lambda^{(i)}\hat{\nu}_k^{(i)} - \hat{\lambda}^{(i)}\nu_k^{(i)})/\Delta\lambda^{(i)}, \bar{m}_k^{(i)} = (\lambda^{(i)}\hat{\rho}_k^{(i)} - \hat{\lambda}^{(i)}\rho_k^{(i)})/\Delta\lambda^{(i)}$, $\bar{b}^{(i)} = (\lambda^{(i)}\hat{\sigma}^{(i)} - \hat{\lambda}^{(i)}\sigma^{(i)})/\Delta\lambda^{(i)}, \bar{m}^{(i)} = (\lambda^{(i)}\hat{\eta}^{(i)} - \hat{\lambda}^{(i)}\eta^{(i)})/\Delta\lambda^{(i)}$, $\bar{s}^{(i)} = (\lambda^{(i)}\hat{\psi}^{(i)} - \hat{\lambda}^{(i)}\psi^{(i)})/\Delta\lambda^{(i)}$. Then, we plug these into the equations (2) to obtain

$$\begin{aligned}
&\mu_k^{(i)} = \bar{u}_k^{(i)} + \lambda^{(i)}\gamma_k, \quad \hat{\mu}_k^{(i)} = \bar{u}_k^{(i)} + \hat{\lambda}^{(i)}\gamma_k \\
&\nu_k^{(i)} = \bar{v}_k^{(i)} + \lambda^{(i)}\tau_k, \quad \hat{\nu}_k^{(i)} = \bar{v}_k^{(i)} + \hat{\lambda}^{(i)}\tau_k \\
&\rho_k^{(i)} = \bar{m}_k^{(i)} + \lambda^{(i)}\xi_k, \quad \hat{\rho}_k^{(i)} = \bar{m}_k^{(i)} + \hat{\lambda}^{(i)}\xi_k \\
&\sigma^{(i)} = \bar{b}^{(i)} + \lambda^{(i)}\theta^{(i)}, \quad \hat{\sigma}^{(i)} = \bar{b}^{(i)} + \hat{\lambda}^{(i)}\theta^{(i)} \\
&\eta^{(i)} = \bar{m}^{(i)} + \lambda^{(i)}\zeta^{(i)}, \quad \hat{\eta}^{(i)} = \bar{m}^{(i)} + \hat{\lambda}^{(i)}\zeta^{(i)} \\
&\psi^{(i)} = \bar{s}^{(i)} + \lambda^{(i)}\delta^{(i)}, \quad \hat{\psi}^{(i)} = \bar{s}^{(i)} + \hat{\lambda}^{(i)}\delta^{(i)} \\
&c_{k+1}^{(i)} = c_k^{\theta^{(i)}} \widetilde{h}^{\bar{t}_k^{(i)}} = g^{\theta^{(i)(k+1)}} \widetilde{h}^{\xi_{k+1}^{(i)}}, \widetilde{h}^{\eta^{(i)}} = \widetilde{C}^{(i)}(c_{\bar{n}}^{(i)}/g)^{\lambda^{(i)}} \\
&M_k^{(i)} = (g^{\bar{u}_k}\widetilde{h}^{\bar{m}_k})^{z_1^{(i)}} (a_k^{\bar{u}_k} g^{\bar{v}_k})^{z_2} (b_k^{\bar{u}_k} h^{\bar{v}_k})^{z_3} \\
&\prod_{k=0}^{\bar{n}-1} (c_k^{(i)})^{y_k \sigma^{(i)}} \widetilde{h}^{\psi^{(i)}} = C^{(i)} \prod_{k=0}^{\bar{n}-1} (c_{k+1}^{(i)})^{y_k \lambda^{(i)}} \\
&g^{z_1^{(i)}\mu_k^{(i)} + z_2\nu_k^{(i)}} \widetilde{h}^{z_1^{(i)}\rho_k^{(i)}} a_k^{z_2\mu_k^{(i)}} b_k^{z_3\mu_k^{(i)}} h^{z_3\nu_k^{(i)}} = M_k^{(i)} (c_k^{(i)})^{z_1\lambda^{(i)}} d_k^{z_2\lambda^{(i)}} e_k^{z_3\lambda^{(i)}} \\
&d_k = a_k^{\gamma_k} g^{\tau_k}, \quad e_k = b_k^{\gamma_k} h^{\tau_k}
\end{aligned} \tag{3}$$

We emphasize that $\gamma_k^{(i)} \neq \theta^{(i)^k}$, otherwise we can extract $\theta^{(i)}$ and $\tau_k^{(i)}$ s.t., $d_k = a_k^{\gamma_k^{(i)}} g^{\tau_k^{(i)}} = a_k^{\theta^{(i)^k}} g^{\tau_k^{(i)}}, e_k = b_k^{\gamma_k^{(i)}} h^{\tau_k^{(i)}} = b_k^{\theta^{(i)^k}} h^{\tau_k^{(i)}}$, which contradicts our original assumption that the instance of $\mathcal{A}$ is incorrect. Now, we show how to use (2) to compute the discrete logarithm of h. We rewrite (2) as

$$\begin{aligned}
g^{z_1^{(1)}\gamma_k^{(1)}+z_2^{(1)}\tau_k^{(1)}} \widetilde{h}^{z_1^{(1)}\xi_k^{(1)}} a_k^{z_2^{(1)}\gamma_k^{(1)}} b_k^{z_3^{(1)}\gamma_k^{(1)}} h^{z_3^{(1)}\tau_k^{(1)}} &= (c_k^{(1)})^{z_1^{(1)}} d_k^{z_2^{(1)}} e_k^{z_3^{(1)}} \\
g^{z_1^{(2)}\gamma_k^{(2)}+z_2^{(2)}\tau_k^{(2)}} \widetilde{h}^{z_1^{(2)}\xi_k^{(2)}} a_k^{z_2^{(2)}\gamma_k^{(2)}} b_k^{z_3^{(2)}\gamma_k^{(2)}} h^{z_3^{(2)}\tau_k^{(2)}} &= (c_k^{(2)})^{z_1^{(2)}} d_k^{z_2^{(2)}} e_k^{z_3^{(2)}} \\
g^{z_1^{(3)}\gamma_k^{(3)}+z_2^{(3)}\tau_k^{(3)}} \widetilde{h}^{z_1^{(3)}\xi_k^{(3)}} a_k^{z_2^{(3)}\gamma_k^{(3)}} b_k^{z_3^{(3)}\gamma_k^{(3)}} h^{z_3^{(3)}\tau_k^{(3)}} &= (c_k^{(3)})^{z_1^{(3)}} d_k^{z_2^{(3)}} e_k^{z_3^{(3)}}
\end{aligned} \tag{4}$$

We should try to eliminate the terms involving a_k, b_k, d_k, and e_k using (4) to obtain two equations. Therefore, we want to ensure that $\{z_l^{(i)} = z_l^{(1)}\}_{l=2,3}$ and $z_1^{(i)} \neq z_1^{(1)}$ for $i = 2,3$. Using the technique in [4], we can define $z_l^{(i)} = H(*, \{a_j, b_j, d_j, e_j\}_{j=0}^{\bar{n}-1})$ where $* \in \{2,3\}$ in one shot, when $\mathcal{A}$ queries the RO for $H(*, \{a_j, b_j, d_j, e_j\}_{j=0}^{\bar{n}-1})$ at the first time. In the same way, we ensure that $\{y_k^{(i)} = z_k^{(1)}\}_{k=0}^{\bar{n}-1}$. For simplicity, we denote $z_l = z_l^{(1)} = z_l^{(2)} = z_l^{(3)}$ for $l = 2,3$ and $y_k = y_k^{(1)} = y_k^{(2)} = y_k^{(3)}$ for $k \in [0, \bar{n}-1]$.

We define $\Delta\gamma_k = \gamma_k^{(1)} - \gamma_k^{(2)}, \Delta'\gamma_k = \gamma_k^{(1)} - \gamma_k^{(3)}, \Delta\tau_k = \tau_k^{(1)} - \tau_k^{(2)}, \Delta'\tau_k = \tau_k^{(1)} - \tau_k^{(3)}, \Delta\xi_k = \xi_k^{(1)} - \xi_k^{(2)}, \Delta'\xi_k = \xi_k^{(1)} - \xi_k^{(3)}$. By using (4), we can obtain

$$\begin{aligned}
g^{z_1^{(1)}\gamma_k^{(1)} - z_1^{(2)}\gamma_k^{(2)} + z_2\delta\tau_k^{(1)}} \widetilde{h}^{z_1^{(1)}\xi_k^{(1)} - z_1^{(2)}\xi_k^{(2)}} (a_k^{z_2} b_k^{z_3})^{\delta\gamma_k^{(1)}} h^{z_3\delta\tau_k^{(1)}} &= \frac{(c_k^{(1)})^{z_1^{(1)}}}{(c_k^{(2)})^{z_1^{(2)}}} \\
g^{z_1^{(1)}\gamma_k^{(1)} - z_1^{(3)}\gamma_k^{(3)} + z_2\delta\tau_k^{(2)}} \widetilde{h}^{z_1^{(1)}\xi_k^{(1)} - z_1^{(3)}\xi_k^{(3)}} (a_k^{z_2} b_k^{z_3})^{\delta\gamma_k^{(2)}} h^{z_3\delta\tau_k^{(2)}} &= \frac{(c_k^{(1)})^{z_1^{(1)}}}{(c_k^{(3)})^{z_1^{(3)}}}
\end{aligned} \tag{5}$$

For all k, we substitute $c_k^{(i)} = g^{\theta^{(i)^k}} \widetilde{h}^{\xi_k^{(i)}}$ into (5), then

$$\begin{aligned}
g^{z_1^{(1)}(\gamma_k^{(1)} - \theta^{(1)^k}) - z_1^{(2)}(\gamma_k^{(2)} - \theta^{(2)^k}) + z_2\delta\tau_k^{(1)}} h^{z_3\delta\tau_k^{(1)}} &= (a_k^{z_2} b_k^{z_3})^{-\delta\gamma_k^{(1)}} \\
g^{z_1^{(1)}(\gamma_k^{(1)} - \theta^{(1)^k}) - z_1^{(3)}(\gamma_k^{(3)} - \theta^{(3)^k}) + z_2\delta\tau_k^{(2)}} h^{z_3\delta\tau_k^{(2)}} &= (a_k^{z_2} b_k^{z_3})^{-\delta\gamma_k^{(2)}}
\end{aligned} \tag{6}$$

We substitute equations (3) into each other and we can obtain $g^{z_1^{(i)}(\gamma_k^{(i)} - \theta^{(i)^k})} a_k^{z_2(\gamma_k^{(i)} - \theta^{(i)^k})} b_k^{z_3(\gamma_k^{(i)} - \theta^{(i)^k})} = 1$ for $i = 1,2,3$. Define $\kappa_1 = g^{z_1^{(1)}} a_k^{z_2} b_k^{z_3}$, $\kappa_2 = g^{z_1^{(2)}} a_k^{z_2} b_k^{z_3}$ and $\kappa_3 = g^{z_1^{(3)}} a_k^{z_2} b_k^{z_3}$. Then we obtain $g^{z_1^{(1)} - z_1^{(2)}} = \kappa_1/\kappa_2, g^{z_1^{(1)} - z_1^{(3)}} = \kappa_1/\kappa_3, g^{z_1^{(2)} - z_1^{(3)}} = \kappa_2/\kappa_3$ and

$$\begin{aligned}
g^{(z_1^{(1)} - z_1^{(2)})(\gamma_k^{(1)} - \theta^{(1)^k})(\gamma_k^{(2)} - \theta^{(2)^k})} &= 1 \\
g^{(z_1^{(1)} - z_1^{(3)})(\gamma_k^{(1)} - \theta^{(1)^k})(\gamma_k^{(3)} - \theta^{(3)^k})} &= 1 \\
g^{(z_1^{(2)} - z_1^{(3)})(\gamma_k^{(2)} - \theta^{(2)^k})(\gamma_k^{(3)} - \theta^{(3)^k})} &= 1
\end{aligned}$$

Recall that $z_1^{(1)} \neq z_1^{(2)}, z_1^{(1)} \neq z_1^{(3)}$. Therefore, $\exists$ two different $i \in [1,3]$ s.t., $\gamma_k^{(i)} - \theta^{(i)^k} = 0$, which implies that, we have extracted the required witness $\theta^{(i)}$ and $\tau_k^{(i)}$ s.t., $d_k = a_k^{\gamma_k^{(i)}} g^{\tau_k^{(i)}} = a_k^{\theta^{(i)^k}} g^{\tau_k^{(i)}}, e_k = b_k^{\gamma_k^{(i)}} h^{\tau_k^{(i)}} = b_k^{\theta^{(i)^k}} h^{\tau_k^{(i)}}$, which contradicts the premise assumption that the instance is invalid.

From (6), we can obtain that if there exists $k \in [1, \bar{n}-1]$ s.t., $\Delta\gamma_k \neq 0, \Delta'\gamma_k \neq 0$ and $\frac{\Delta\tau_k}{\Delta'\tau_k} \neq \frac{\Delta\gamma_k}{\Delta'\gamma_k}$, we can obtain the solution of the DL problem instance $\log_g h = \frac{z_2\Delta\tau_k/\Delta\gamma_k - z_2\Delta'\tau_k/\Delta'\gamma_k}{z_3\Delta'\tau_k/\Delta'\gamma_k - z_3\Delta\tau_k/\Delta\gamma_k}$.

Else if $\forall\ k \in [1, \bar{n}-1]$, $\Delta\gamma_k = 0$, and $\exists\ k \in [1, \bar{n}-1]$ s.t., $\Delta\tau_k \neq 0$, we can compute the solution of the DL problem instance $\log_g h = -w_k/(z_3\Delta\tau_k)$.

Else, there are other two cases that we cannot compute the solution of DL problem instance: 1) $\forall\ k \in [1, \bar{n}-1]$ s.t., $\Delta\gamma_k = \Delta\tau_k = \Delta'\gamma_k = \Delta'\tau_k = 0$; 2) for all k satisfying $\Delta\gamma_k \neq 0, \Delta'\gamma_k \neq 0$, $\frac{\Delta\tau_k}{\Delta'\tau_k} = \frac{\Delta\gamma_k}{\Delta'\gamma_k}$.

Now we show that neither of these cases can happen. Case 1) can be excluded because the instances are not identical. Assume that $\gamma_k^{(2)} = x_k^{(1)}\gamma_k^{(1)}, \gamma_k^{(3)} = x_k^{(2)}\gamma_k^{(1)}$. In case 2), we have $\frac{\Delta\tau_k}{\Delta'\tau_k} = \frac{\Delta\gamma_k}{\Delta'\gamma_k} = \frac{x_k^{(1)} - 1}{x_k^{(2)} - 1}$. We use (4) to obtain $g^{(z_1^{(2)}\gamma_k^{(1)} - z_1^{(1)}\gamma_k^{(1)})x_k^{(1)}} \widetilde{h}^{z_1^{(2)}\xi_k^{(2)} - z_1^{(1)}\xi_k^{(1)}x_k^{(1)}} = (d_k^{z_2} e_k^{z_3})^{1 - x_k^{(1)}} (c_k^{(2)})^{z_1^{(2)}} / ((c_k^{(1)})^{z_1^{(1)}x_k^{(1)}})$, then we have $g^{(z_1^{(2)}\gamma_k^{(1)} - z_1^{(1)}\gamma_k^{(1)})x_k^{(1)}} = g^{(z_1^{(2)}\theta^{(2)^k} - z_1^{(1)}x_k^{(1)}\theta^{(2)^k})} (d_k^{z_2} e_k^{z_3})^{1 - x_k^{(1)}}$.

Recall that $\exists$ two different $i \in [1,3]$ s.t., $\gamma_k^{(i)} - \theta^{(i)^k} = 0$. Without losing generality, we assume that this condition is satisfied when i is 1 and 2. Then we have $\gamma_k^{(2)} = x_k^{(1)}\gamma_k^{(1)} = \theta^{(2)^k} = x_k^{(1)}\theta^{(1)^k}$. Thus, there exists $w^{(1)}$ s.t., $x_k^{(1)} = w^{(1)^k}$. Thus, we can obtain $(d_k^{z_2} e_k^{z_3})^{1 - w^{(1)^k}} = 1$ for all $k \in [1, \bar{n}-1]$. For $w^{(1)} \neq 1$, it is difficult for $\mathcal{A}$ to generate $\{d_k, e_k\}_{k=0}^{\bar{n}-1}$ satisfying $(d_k^{z_2} e_k^{z_3})^{1 - w^{(1)^k}} = 1$ due to the collision resistance of hash function. So $w^{(1)} = 1, \Delta\gamma_k = 0$, which contradicts the assumption in case 2). Hence we proved that the solution of the DL problem instance is valid. The success probability bound is given by the multiple-forking lemma with $n = 5$.

Honest Verifier Zero-Knowledge. Recall that this property is satisfied if $\exists$ a polynomial-time simulator which can simulate the view of an honest verifier interacting with an honest prover. We now construct a simulator S as follows: when given the public parameters $g, h, \tilde{h}, \{a_k, b_k, d_k, e_k\}_{k=0}^{\bar{n}-1}$, S chooses random $\lambda, \sigma, \eta, \psi, \{c_k\}_{k=1}^{n}$ and $\{\mu_k, \nu_k, \rho_k\}_{k=0}^{\bar{n}-1}$ according to the appropriate distribution and computes $\widetilde{C} = \tilde{h}^{\eta}(c_{\bar{n}}/g)^{-\lambda}, C = (\prod_{k=0}^{\bar{n}-1} c_{k+1}^{y_k})^{\lambda} (\prod_{k=0}^{\bar{n}-1} c_k^{y_k})^{-\sigma} \tilde{h}^{-\psi}, M_k = c_k^{z_1\lambda} d_k^{z_2\lambda} e_k^{z_3\lambda} g^{-z_1\mu_k - z_2\nu_k} \widetilde{h}^{-z_1\rho_k} a_k^{-z_2\mu_k} b_k^{-z_3\mu_k} h^{-z_3\nu_k}$, where $z_i = H(i, \{a_j, b_j, d_j, e_j\}_{j=0}^{\bar{n}-1})$ for $i = 1,2,3$ and $y_k = H(\{a_j, b_j, d_j, e_j\}_{j=0}^{\bar{n}-1}, k)$. It is easy to see that the distribution of the transcript $(\widetilde{C}, \{c_{k+1}, M_k\}_{k=0}^{\bar{n}-1}, C, \lambda, \sigma, \eta, \psi, \{\mu_k, \nu_k, \rho_k\}_{k=0}^{\bar{n}-1})$ is identical to the distribution of the view of an honest verifier

interacting with the prover. □

C.2 Proof of Theorem 2

Proof. We construct a PPT simulator $\mathcal{S}$ given access to the functionality $\mathcal{F}_{\text{E2L}}$ running the PPT adversary $\mathcal{A}$ as a subroutine. Specifically, the simulation is constructed as follows: 1) $\mathcal{S}$ simulates Π_{Log} for each $i \in \mathbf{A}$ and $j \in \mathbf{H}$ to generate all key pairs (pk_i, sk_i) and sends the relevant parts to the corresponding party. 2) $\mathcal{S}$ samples $\langle x \rangle_j^a \leftarrow \mathcal{M}$ for $j > 1 \wedge j \in \mathbf{H}$, computes l_j, cip_j and simulates Π_{Ran} for parties $\{P_j\}_{j>1, j\in\mathbf{H}}$, receives $\{l_i, cip_i\}_{i>1, i\in\mathbf{A}}$ and extracts $\{\langle x \rangle_i^a\}_{i>1, i\in\mathbf{A}}$ from Π_{Ran} sent by $\mathcal{A}$. 3) If P_1 is an honest party, $\mathcal{S}$ performs the original protocol steps with simulating Π_{Ran}. If P_1 is a corrupted party, $\mathcal{S}$ receives l_1, cip_1 with a proof Π_{Ran} from $\mathcal{A}$ and extracts $\langle x \rangle_1^a$ from Π_{Ran}. If $\langle x \rangle_1^a \neq \log_g(c_0^{-sk_1} g^{(n-1)p} c_1 \prod_{i=2}^n l_i) \bmod p$ or $l_1 \neq g^{-\langle x \rangle_1^a} c_0^{-sk_1}$ or $cip_1 \neq g^{\langle x \rangle_1^a} h^{sk_1}$, $\mathcal{S}$ sets fail and aborts.

The simulation of $\mathcal{F}_{\text{E2L}}$ is perfect. Because the ElGamal-THE scheme is CPA secure, the simulation using random shares for honest parties is computationally indistinguishable from the real protocol execution. Since Π_{Ran} possess special soundness, $\mathcal{A}$ can not provide cip_1, Π_{Ran} for $\langle x \rangle_1^a$ without satisfying $x = \sum_{i=1}^n \langle x \rangle_i^a$ in step 3) to pass the verification. Furthermore, the correctness of the sum of the shares from all parties can always be check, ensuring correctness. In summary, the adversary's view and the outputs of the honest parties are perfectly indistinguishable in both the real and ideal worlds. Therefore, the protocol Π_{E2L} securely realizes $\mathcal{F}_{\text{E2L}}$. □

C.3 Proof of Theorem 3

Proof. We define a shorthand for expressing ElGamal ciphertexts over vectors. For a scalar base $g \in \mathbb{G}$ and vector $\vec{a} = (a_0, \ldots, a_{M-1}) \in \mathbb{Z}_q^M$, we write $\vec{a} \circ g := (g^{a_0}, \ldots, g^{a_{M-1}})$. For vectors $\vec{a}, \vec{b} \in \mathbb{Z}_q^M$, and group bases $g, h \in \mathbb{G}$, we define $\vec{a} \circ g + \vec{b} \circ h := (g^{a_0} h^{b_0}, \ldots, g^{a_{M-1}} h^{b_{M-1}})$, where group multiplication encodes additive plaintexts. We also extend $\circ$ to vector bases: for $\vec{\beta} \in \mathbb{Z}_q^M$ and $\vec{g} \in \mathbb{G}^M$, define $\vec{\beta} \circ \vec{g} := (g_0^{\beta_0}, \ldots, g_{M-1}^{\beta_{M-1}})$. This formalism simplifies the notation for homomorphic rotations and DFT-encoded encryptions. Moreover, for matrix $A \in \mathbb{Z}_q^{M \times M}$ and vectors $\vec{a}, \vec{b}$, we have $A\vec{a} \circ g + A\vec{b} \circ h = A \otimes (\vec{a} \circ g + \vec{b} \circ h)$, where $A \otimes (\cdot)$ denotes row-wise group multiplication, preserving encryption structure under linear transforms. Assume all n parties follow the protocol honestly in the $\mathcal{F}_{\text{Con}}$-hybrid model. Let $T = \vec{x} \in (\mathbb{Z}_q)^M$ denote the table to be masked.

1) Initial encryption and encoding. In Case 1 (public table), party P_1 computes ElGamal encryption of each entry in $\vec{x}$ using TEnc, obtaining $[\![T]\!]$, then applies DFT to get $[\![T_0]\!] = \mathsf{DFT}(\bar{n}, \alpha, [\![T]\!]) = (A \otimes (\vec{r}_0 \circ g),\ A \otimes (\vec{x} \circ g + \vec{r}_0 \circ h)) = (\vec{a}, \vec{b})$, where A is the DFT matrix, and $\vec{r}_0$ is the vector of encryption randomness. In Case 2 (private table), P_1 aggregates encrypted shares $\sum_i \mathbf{c}_i$ and similarly computes $[\![T_0]\!]$ via DFT.

2) Cascading masked rotations. Each party P_i samples rotation parameter $r_i \in \mathbb{Z}_M$, defines $\vec{\beta}_i = (\alpha^{0 \cdot r_i}, \ldots, \alpha^{(M-1) \cdot r_i})$, and blinding vector $\vec{s}_i \in \mathbb{Z}_q^M$. It then homomorphically rotates and blinds the ciphertext as $[\![T_i]\!] = (\vec{a}_i, \vec{b}_i) = (\vec{\beta}_i \circ \vec{a}_{i-1} + \vec{s}_i \circ g,\ \vec{\beta}_i \circ \vec{b}_{i-1} + \vec{s}_i \circ h)$. Expanding recursively, we obtain: $\vec{a}_n = \vec{\beta}_{1..n} \circ \vec{a} + \sum_{j=1}^n \left(\vec{\beta}_{j+1..n} \circ \vec{s}_j \circ g\right)$ and $\vec{b}_n = \vec{\beta}_{1..n} \circ \vec{b} + \sum_{j=1}^n \left(\vec{\beta}_{j+1..n} \circ \vec{s}_j \circ h\right)$, where $\vec{\beta}_{j..k} := \vec{\beta}_k \circ \cdots \circ \vec{\beta}_j$.

3) Inverse transform and share extraction. The final ciphertext $[\![T_n]\!]$ is decrypted by applying IDFT: $[\![T']\!] = \mathsf{IDFT}(\bar{n}, \alpha, [\![T_n]\!]) = (A^{-1} \otimes \vec{a}_n,\ A^{-1} \otimes \vec{b}_n)$. Each blinding term of the form $A^{-1} \otimes (\vec{\beta}_{j+1..n} \circ \vec{s}_j \circ g)$ is independent of $\vec{x}$ and cancels during ElGamal decryption. Therefore, the decryption yields $T' =$ rotation of T by $r = \sum_{i=1}^n r_i \bmod M$. The parties then convert $[\![T']\!]$ into additive shares $\langle T' \rangle^a$ using the conversion functionality $\mathcal{F}_{\text{Con}}$.

All ZKPoKs ensure integrity of inputs, rotations, and commitments. Under honest behavior, no step fails, and the protocol correctly outputs $\langle r \rangle^a, \langle T' \rangle^a, \{c_{r_i}, [\![\langle T' \rangle_i^a]\!]\}_{i=1}^n$. □

C.4 Proof of Theorem 4

Proof. We construct a PPT simulator $\mathcal{S}$ given access to the functionality $\mathcal{F}_{\text{Prep-LUT}}$ running the PPT adversary $\mathcal{A}$ as a subroutine, and emulates $\mathcal{F}_{\text{E2L}}$. Specifically, the simulation is constructed as follows:

1) $\mathcal{S}$ simulates Π_{Log} for each $i \in \mathbf{A}$ and $j \in \mathbf{H}$ to generate all key pairs (pk_i, sk_i) and sends the relevant parts to the corresponding party.

2) For each honest party P_j $(j \in \mathbf{H})$, $\mathcal{S}$ samples $r_j \leftarrow_R \mathbb{Z}_M, r'_j \leftarrow_R \mathbb{Z}_q$, and computes $\beta_j = \alpha^{r_j}, c_{r_j} = (g^{r'_j}, g^{r_j} h^{r'_j})$.

3) If P_1 is honest, $\mathcal{S}$ follows the protocol to compute $[\![T]\!]$.

4) From 1 to n: If P_i is honest, $\mathcal{S}$ chooses r_i randomly, computes $[\![T_i]\!]$ by rotating $[\![T_{i-1}]\!]$ with r_i and simulates Π_{Cip} and Π_{Rot}; For $i < n$, if P_{i-1} is corrupted and P_i is honest, $\mathcal{S}$ receives $[\![T_{i-1}]\!], c_{r_i}, \Pi_{\text{Cip}}, \Pi_{\text{Rot}}$ from $\mathcal{A}$ and extracts r_i, r'_i, and broadcasts $[\![T_i]\!], \Pi_{\text{Cip}}, \Pi_{\text{Rot}}$; If P_n is honest, $\mathcal{S}$ broadcasts $[\![T_n]\!], \Pi_{\text{Cip}}, \Pi_{\text{Rot}}$.

5) $\mathcal{S}$ emulates $\mathsf{E2L}$ of $\mathcal{F}_{\text{Con}}$ to generate $\langle T' \rangle^a$ and $[\![\langle T' \rangle_i^a]\!]$ by recording the shares of corrupted parties about $\langle T' \rangle^a$ and $[\![\langle T' \rangle_i^a]\!]$ sent by $\mathcal{A}$ to $\mathcal{F}_{\text{Con}}$.

6) If $\sum_{i=1}^n r_i$ is not the plaintext of $\sum_{i=1}^n c_i$ using encrypt random salts $\sum_{i=1}^n r'_i$, $\mathcal{S}$ sets fail and aborts.

The simulation of $\mathcal{F}_{\text{Prep-LUT}}$ is perfect. Owing to the CPA security of ElGamal-THE, the use of random values for honest parties yields a view computationally indistinguishable from that of the real protocol. The special soundness of $\Pi_{\text{Cip}}, \Pi_{\text{Rot}}$ ensures that $\mathcal{A}$ cannot produce invalid ciphertexts or ZKPoKs. Moreover, the sum of shares from all parties is verifiable, guaranteeing correctness. So the adversary's view and the outputs of honest parties are indistinguishable in both the real

and ideal worlds. Thus, protocol $\Pi_{\text{Prep-LUT}}$ securely realizes $\mathcal{F}_{\text{Prep-LUT}}$. □

C.5 Proof of Theorem 5

Proof. Assume all n parties behave honestly and inputs are valid. In Step 1, the parties homomorphically add ciphertexts and compute: $c = \sum_{i=1}^{n}(c_{x_i} + c_{r_i}) = \mathsf{TEnc}(x+r)$. The decryption yields $u = x + r \bmod M$. Parties locally reconstruct their shares $\langle u' \rangle_i^a = x_i + r_i$, and verify that $\sum_i \langle u' \rangle_i^a = u$. Since $T'[j] = T[(j-r) \bmod M]$, it follows $T'[u] = T[(x+r-r) \bmod M] = T[x]$. Thus, in Step 2, each party outputs its additive share of $T'[u]$, which equals $\langle T[x] \rangle^a$ by construction. Therefore, the protocol faithfully reconstructs $T[x]$ under correct decryption and honest share addition. □

C.6 Proof of Theorem 6

Proof. We construct a PPT simulator $\mathcal{S}$ given access to the functionality $\mathcal{F}_{\text{LUT}}$ running the PPT adversary $\mathcal{A}$ as a subroutine, and emulates $\mathcal{F}_{\text{Prep-LUT}}$. Specifically, the simulation is constructed as follows: 1) $\mathcal{S}$ simulates Π_{Log} for each $i \in \mathbf{A}$ and $j \in \mathbf{H}$ to generate all (pk_i, sk_i) and sends the relevant parts to the corresponding party. 2) $\mathcal{S}$ emulates $\mathcal{F}_{\text{Prep-LUT}}$ to generate a masked table $(\langle r \rangle^a, \langle T' \rangle^a)$ with ciphertexts $\{c_{r_i}, [\![\langle T' \rangle_i^a]\!]\}_{i=1}^n$ by recording the shares of corrupted parties about $(\langle r \rangle^a, \langle T' \rangle^a)$ and $\{c_{r_i}, [\![\langle T' \rangle_i^a]\!]\}_{i=1}^n$ sent by $\mathcal{A}$ to $\mathcal{F}_{\text{Prep-LUT}}$. 3) $\mathcal{S}$ computes $c = \sum_{i=1}^{n}(c_{r_i} + c_{x_i})$. $\mathcal{S}$ parses c as c_0, c_1. For each honest P_j ($j \in \mathbf{H}$), $\mathcal{S}$ computes $t_j = c_0^{sk_j}$ and simulates Π_{Exp}. For each $i \in \mathbf{A}$, $\mathcal{S}$ receives t_i with related Π_{Exp} from $\mathcal{A}$ and extracts $\{sk_i'\}_{i \in \mathbf{A}}$. If $\exists\ i \in \mathbf{A}$ s.t. $sk_i' \neq sk_i$, $\mathcal{S}$ sets fail and aborts. $\mathcal{S}$ computes $u = \log_g c_1 / (\prod_{i=1}^{n} t_i)$.

The simulation of $\mathcal{F}_{\text{LUT}}$ is perfect. Since Π_{Exp} has special soundness, $\mathcal{A}$ cannot provide invalid ciphertexts or ZKPoKs that pass the verification. The correctness of the sum of shares from all parties is always verifiable, ensuring correctness. The distributions of the adversary's view and the honest parties' outputs are indistinguishable in both the real and ideal worlds. So Π_{LUT} securely realizes $\mathcal{F}_{\text{LUT}}$. □

C.7 Proof of Theorem 7

Proof. We construct a PPT simulator $\mathcal{S}$ in the $\mathcal{F}_{\text{A-MPC}}$-hybrid model. From each corrupted party $P_i \in \mathbf{A}$, $\mathcal{S}$ extracts x_i and r_i from the broadcast ciphertexts c_{x_i} and c_{r_i} using the extractors guaranteed by the ZKPoKs, and forwards the adversarys input to $\mathcal{F}_{\text{Con}}$. The simulator samples a random $u \leftarrow_R \mathbb{Z}_q$ and simulates honest parties ciphertexts $\{c_{x_j}, c_{r_j}\}_{j \in \mathbf{H}}$ such that the aggregated ciphertext $c = \sum_{i=1}^{n}(c_{x_i} + c_{r_i})$ encrypts u, by defining honest contributions satisfying $\sum_{j \in \mathbf{H}}(\tilde{x}_j + \tilde{r}_j) = u - \sum_{i \in \mathbf{A}}(x_i + r_i)$. The simulator then simulates the honest parties threshold decryption messages so that the joint decryption (and hence Open) yields u. Indistinguishability follows from the CPA security of THE, the uniformity of u due to the random mask r, and the special soundness of the ZKPoKs. Therefore, Π_{A2L} securely realizes $\mathsf{A2L}$. □

C.8 Proof of Theorem 8

Proof. We construct a PPT simulator $\mathcal{S}$ in the $\mathcal{F}_{\text{A-MPC}}$-hybrid model. For each corrupted party $P_i \in \mathbf{A}$, $\mathcal{S}$ extracts the committed randomness share r_i from the broadcast ciphertext c_{r_i} and forwards the adversarys effective input contribution implied by the LUT shares to $\mathcal{F}_{\text{Con}}$. The simulator samples $u \leftarrow_R \mathbb{Z}_q$ and simulates honest parties ciphertexts $\{c_{x_j}, c_{r_j}\}_{j \in \mathbf{H}}$ such that the aggregated ciphertext $c = \sum_{i=1}^{n}(c_{x_i} + c_{r_i})$ encrypts u, by defining honest contributions satisfying $\sum_{j \in \mathbf{H}}(\tilde{x}_j + \tilde{r}_j) = u - \sum_{i \in \mathbf{A}}(x_i + r_i)$. The simulator simulates honest partial decryption messages so that the joint decryption yields u. Indistinguishability follows from the CPA security of THE, the uniformity of u due to the random mask r, and the special soundness of the ZKPoKs. Therefore, Π_{L2A} securely realizes $\mathsf{L2A}$. □

C.9 Proof of Theorem 9

Proof. We construct a PPT simulator $\mathcal{S}$ in the $(\mathcal{F}_{\text{A-MPC}}, \mathcal{F}_{\text{LUT}})$-hybrid model. $\mathcal{S}$ receives the Boolean shares $\langle x_j \rangle^b$ from the adversary during calls to $\mathcal{F}_{\text{LUT}}$ and forwards them to $\mathcal{F}_{\text{Con}}$ to obtain the adversarys arithmetic output share contribution. To simulate the execution, $\mathcal{S}$ emulates the output of $\mathcal{F}_{\text{LUT}}$ by generating arithmetic shares of the bits $\langle x_j \rangle^a$ consistent with the ideal output and invokes the simulator of Π_{A2L} for each bit. The final arithmetic share $\langle x \rangle^{\bar{a}}$ is computed locally as $\sum_{j=0}^{l-1} 2^j \langle x_j \rangle^{\bar{a}}$. Since the adversary is semi-honest and all local computations are deterministic, the simulated view is indistinguishable from the real execution by the security of the underlying secret sharing and the hybrid functionalities. □

C.10 Proof of Theorem 10

Proof. We construct a PPT simulator $\mathcal{S}$ in the $(\mathcal{F}_{\text{LUT}}, \mathcal{F}_{\text{A-MPC}})$-hybrid model. $\mathcal{S}$ simulates the generation of random Boolean shares $\langle r \rangle^b$ and the execution of Π_{B2A}. To simulate the opening of $\langle x - r \rangle^{\bar{a}}$, $\mathcal{S}$ samples a uniform value $u \leftarrow_R \mathbb{F}_q$ and broadcasts it to the adversary, which is indistinguishable since r acts as a one-time pad in the real protocol. $\mathcal{S}$ then computes the bit-decomposition of u and, together with $\langle r \rangle^b$, invokes the simulator for the modulo-addition circuit in $\mathcal{F}_{\text{LUT}}$. Finally, $\mathcal{S}$ ensures that the output shares match the output of $\mathcal{F}_{\text{Con}}$. Indistinguishability follows from the uniformity of r and the security of the hybrid functionalities; thus Π_{A2B} securely realizes $\mathsf{A2B}$. □